强化学习精度定位解决方案v0.1

时间	**2025/12/22**
版本	**v0.1**
作者	**昇腾计算生态使能部 — 郑若伟 张宁 胡晓波 全家豪 郑炜 戴文权 王嘉伟 张浩勇 陈留圣 赵弘毅 方之昊 李浩哲 王凌枫 周洋**

1. 强化学习精度概述

1.1 背景

大模型强化学习的精度问题定位流程复杂，影响精度的指标与因素众多，算法的动态随机性与底层硬件、算子、上层框架、数据流传输等多因素耦合，导致根因难以追溯；本文档旨在提供一套系统化的定位指南，帮助用户快速排查可能影响精度的因素，指导快速定位引发精度问题的根本原因，保障项目落地进度。

1.2 强化学习关键指标

本章节对强化学习中的关键指标进行介绍：

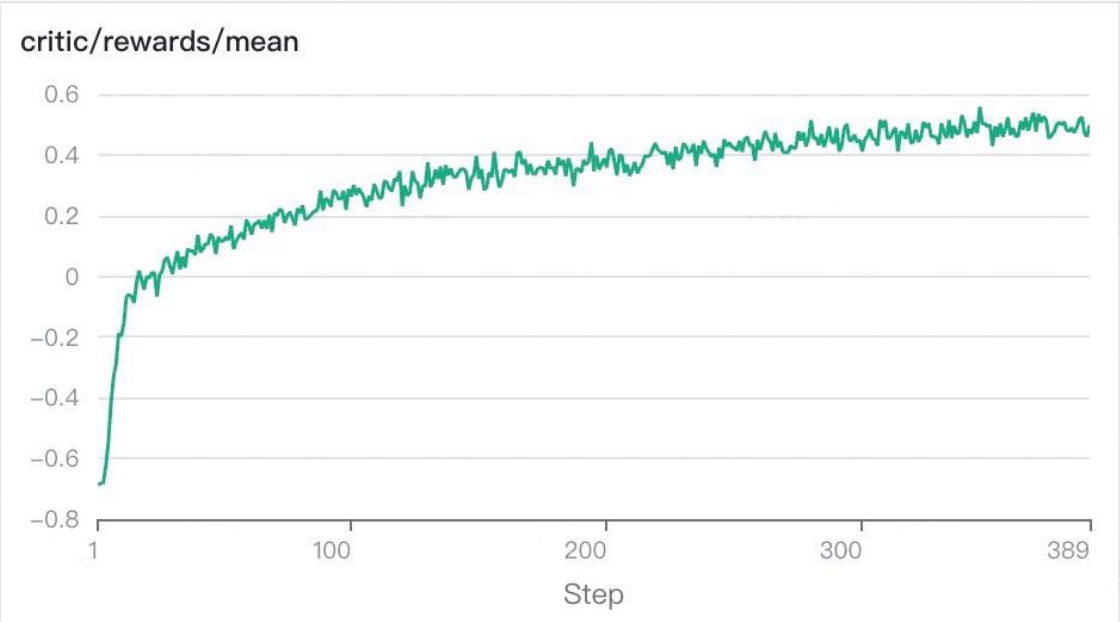
指标名称	含义	数值描述

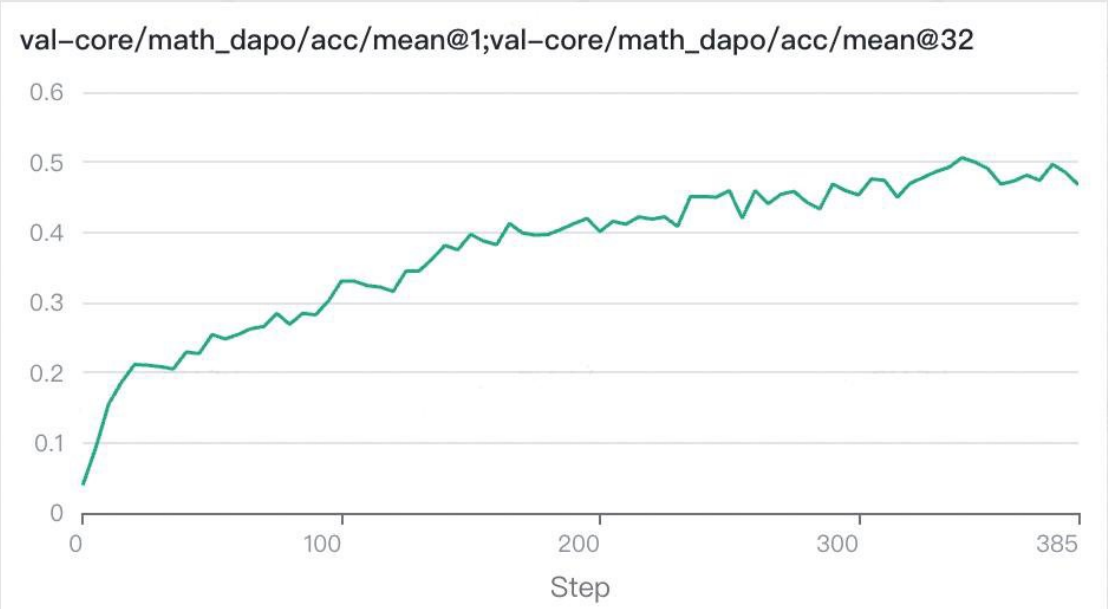
entropy	衡量策略分布的随机程度，越大表示越“随机”、越不确定；越小表示越“确定”、越倾向于贪心选择 在强化学习中,熵(entropy)用于量化策略的不确定性,其数学定义为 策略分布的负对数期望。实际应用中,熵通过以下方式发挥作用:1.平衡探索 与利用:高熵策略鼓励探索未知动作,低熵策略倾向于利用已知最优动作;2. 优化目标函数:许多算法将熵正则项加入目标函数,平衡奖励与探索;3.防止 策略退化:保持一定随机性有助于发现更优路径。	entropy 越大 → 概率分布越均匀 → 模型更“犹豫”、探索更多、输出多样 entropy越小 → 概率分布越尖锐 → 模型倾向于选择少数几个高概率 token，模型趋于稳定，输出更一致 熵坍缩：如果**生成概率分布非常尖锐**（例如一个 token 占 99% 概率，其他 token 占 1%），那么熵会接近 0 → 多样性低 → 可能陷入熵坍缩，过低的熵表示概率分布过于尖锐，导致失去探索能力
pg_loss	策略梯度优化目标的损失值，通常代表新旧策略的偏移量	pg_loss 大：策略更新强烈，说明新旧策略差异大 pg_loss 小：策略更新幅度小，训练趋于稳定 **不同的算法在计算 pg_loss 时有不同粒度：** • Sample - GRPO • Sequence - GSPO • Token - DAPO
response length	模型回复输出的 token 长度	• 可以反映模型是否出现碎片化回复（过短）或是否出现啰嗦回复（过长）

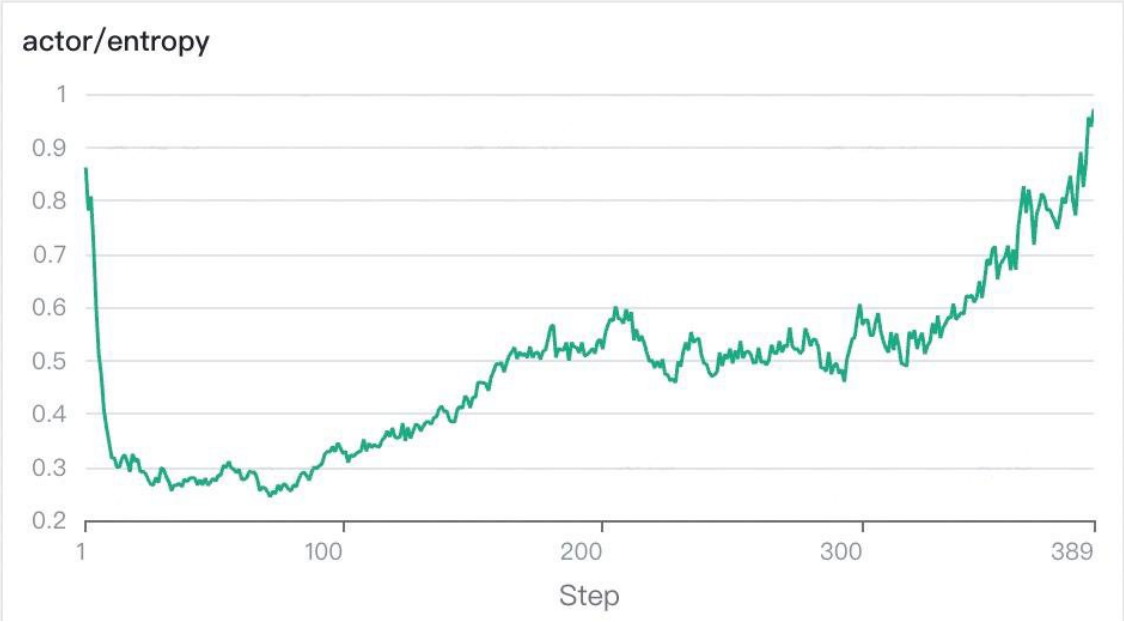
		• response/mean 平均回复长度也可以作为判断性能的相对指标
log_ppl_diff	是量化训推一致性的核心指标，能直接反映训练侧和推理侧模型对同一文本的 “置信度差异” **计算方式：**mean(log_ppl_rollout - log_ppl_training)	正值表示rollout 侧（推理）比训练侧更 “困惑”（log PPL 更高），训练侧对数据更有信心，偏差相对安全，由于采集一致性指标徐额外引入显存占用，默认关闭，开启需要添加如下脚本配置：actor_rollout_ref.rollout.calculate_log_probs=True \
ppl_ratio	训练PPL与rollout PPL的比率 **计算方式：**exp(mean(log(training_ppl / rollout_ppl)))	大于 1.0 表示训练的置信度低于rollout，与上者同为训推一致性指标，开始方式同上
reward	来自奖励模型（RM）的分数，是 RL 优化的核心目标，反映的是“策略偏好”或“基于规则/判别器的质量评价“	reward 越高表示模型输出越能满足训练目标
acc	acc 是基于标准答案的“正确性”	越高表示更接近标签答案
clip_ratio	是 PPO 算法中对**新旧策略动作概率比值**施加截断操作后的结果，用于限制单次策略更新幅度，避免训练发散	数值**接近 1**：新旧策略差异小，更新幅度保守 数值**频繁触达 1±ε**：说明策略更新幅度达到了截断阈值
grad norm	是衡量模型训练时参数梯度向量大小的核心指标，用于量化梯度的整体强度	数值**低**：说明梯度消失，模型参数几乎没有更新 数值**适中且稳定**：梯度更新幅度合理，是模型稳定学习的理想状态 数值**过高**（如突然飙升）：说明梯度爆炸，参数更新幅度过大

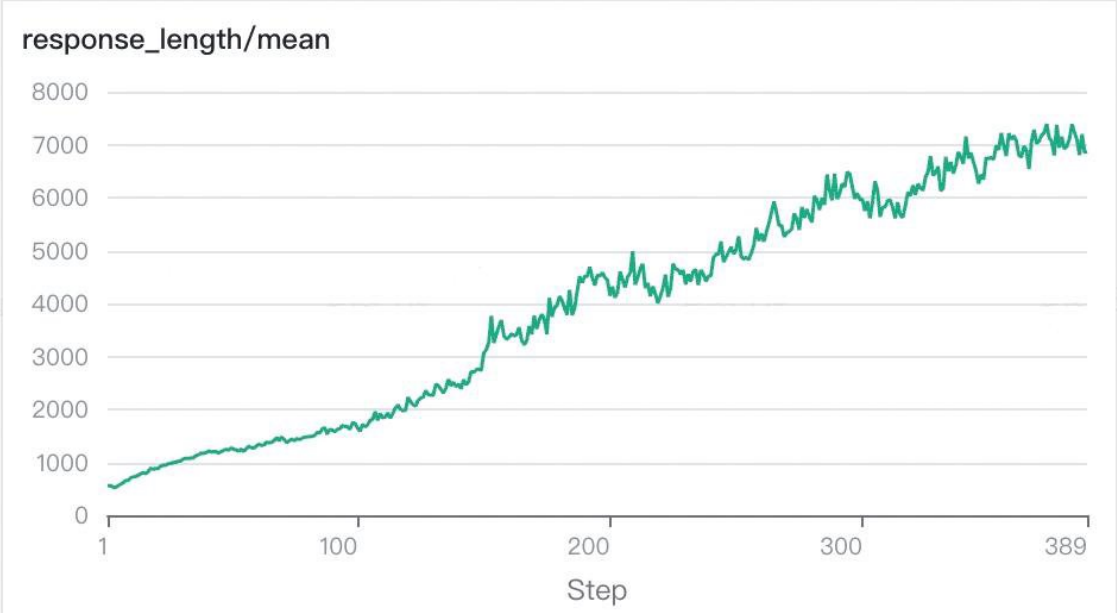
ppo_kl	新旧策略的KL散度（衡量策略更新幅度，过大可能导致训练不稳定）	KL 散度越小，新旧策略越相似； KL 散度越大，策略更新幅度越剧烈 理论上在 PPO 训练的第一步该值等于 **0**，原因是**新旧策略完全相同，**但verl中会显示非0

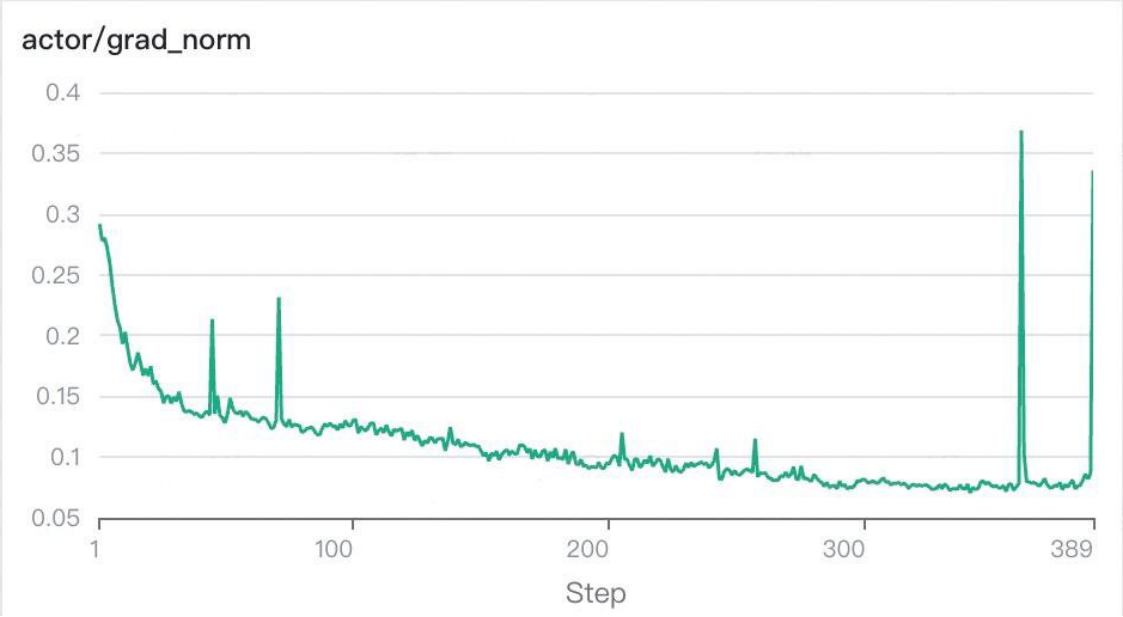
在强化学习训练过程中，部分指标会随时间呈现典型变化， **正常训练情况下，reward 和 acc 会同时上升**，呈现正相关。Qwen2.5-32B模型为例，给出指标的训练曲线参考如下：

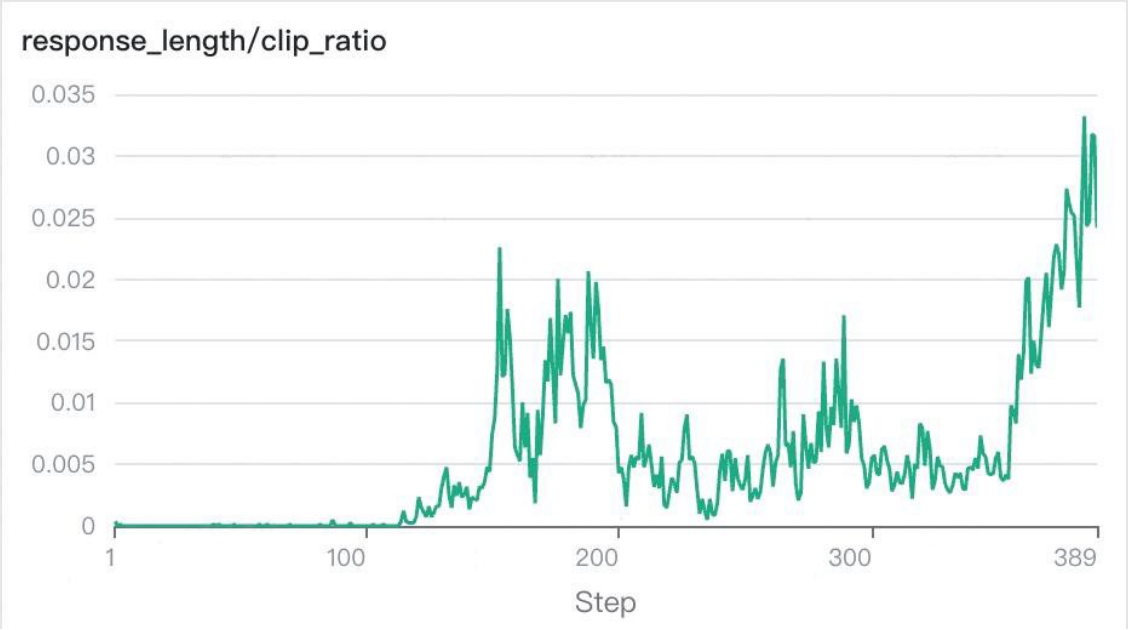





actor/entropy
1
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
1
100
200
300
389
Step


response_length/mean
8000
7000
6000
5000
4000
3000
2000
1000
0
1
100
200
300
389
Step





1.3 精度对齐标准

1.3.1 精度指标

- 下游任务测评得分是首要指标，要求相对误差<2%。
- Reward mean 绝对平均误差，非DAPO算法要求小于4%，DAPO算法要求小于6%。

- 当上述两个指标反馈RL精度出现问题时，可参考过程指标判断RL训练是否输出健康水平，包括Grad Norm误差小于4%，打桩response后loss误差小于1%，首token logp余弦相似度>99.9%等等（非严格要求）。在实际的精度交付中，由于结果指标测试成本大，建议先对齐过程指标，以此检查结果指标的反复实验。

1.3.2 测试方法

强化学习训练耗时不一，有些任务单step近似于普通训练任务，但是例如DAPO长序列等任务一个step可能需要消耗几个小时，因此这里在权衡测试结果的采样覆盖度和效率上，建议测试时需要**保证24小时且跑完100步以上**。两个条件需要同时满足。对于数据集规模较小的测试任务，如果时间可接受，应该以跑完任务为准。

1.4 精度日志结果查看

方法1：通过上传log文件，离线解析：

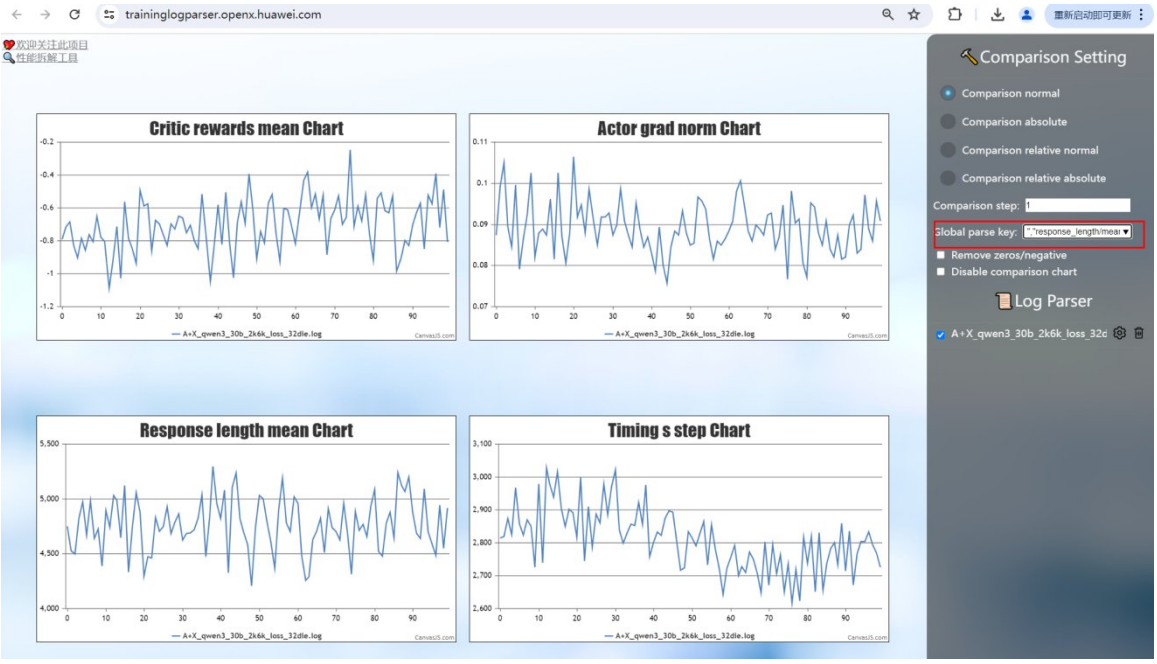
使用方法：打开如下网站，上传文件，根据自己需要的指标值进行自动解析

内部绿区网站：https://traininglogparser.openx.huawei.com/

外部蓝区网站：https://curryrice233.github.io/TrainingLogParser/

示例：使用内部网站，基于红框配置自己的指标值关键词获取相应的结果。

网站已配置可选关键词：`["critic/rewards/mean:","actor/grad_norm:","actor/kl_loss:","response_length/mean:"]`，如果需要增加，在中括号内添加键值即可。



方法2：训练启动前接入wandb的服务信息，以自动展示相关结果，wandb服务可分为云端使用（官方托管）和本地 / 私有部署（Self-hosted）两种，一般客户自行接入。

注意：wandb涉及云端数据上传，绿区不建议使用。

如下是基于verl仓库做wandb接入的配置参考，需要在训练启动sh脚本中设置密钥和wandb服务的配置

- 设置WANDB API密钥环境变量，用于识别身份，需要在拉起ray之前设置：
  `export WANDB_API_KEY=<your-wandb-api-key>`
- 训练参数配置，明确使用wandb服务
  `trainer.logger='["console","wandb"]'`
  `trainer.project_name='verl_example'`
  `trainer.experiment_name='Qwen2.5-32B-Instruct_function_rm'`
- wandb代理配置，参考官方文档：https://github.com/volcengine/verl/blob/main/docs/faq/faq.rst
  `+trainer.wandb_proxy=http://<your proxy and port>`

2. 精度比对前置动作

2.1 精度对齐checklist

本章节梳理了强化学习影响精度相关的代码、环境、模型、数据处理、关键参数等配置清单和验证方法，按照全局到局部的视角顺序给出可实践落地的精度对齐前检查 checklist 参考。

2.1.1代码与环境对齐

NPU强化学习训练的依赖环境总结分为三大类：

1. CANN软件环境：ascend-toolkit、nnal等；
2. 关键第三方库：torch、torch-npu等；
3. 训练/推理框架：VeRL、vllm、vllm-ascend等。

从全局角度来看，应首先确认代码和环境，避免版本差距过大导致的特性不匹配。

NPU和GPU进行精度对齐时，一般来说，**VeRL、transformers版本需要进行强对齐，否则会直接影响精度结果**。其他代码由于NPU版本迭代存在部分滞后性，如无法进行强对齐，需优先保持一致或相近。

	相关环境	使用平台	验证方法

CANN环境	ascend-toolkit、kernel、nnal等	NPU	1、查看环境变量中使用的CANN路径
			`env | grep ASCEND_TOOLKIT_HOME`
			若无输出，说明环境没设置好，需进行set_env
			2、查看CANN版本信息
			`cat /usr/local/Ascend/CANN具体路径/version.cfg`
			建议：优先选择无已知问题的版本

关键第三方库	torch	NPU、GPU	`pip show torch`
	torch-npu	NPU	`pip show torch_npu`
	transformers	NPU、GPU	`pip show transformers`

<table>
<tr><td rowspan="2"></td><td>megatron</td><td>NPU、GPU</td><td>pip show megatron</td></tr>
<tr><td>tensordict</td><td>NPU、GPU</td><td>pip show tensordict

注意：NPU上存在低版本未适配导致精度问题，建议升级至tensordict>=0.10.0</td></tr>
<tr><td rowspan="3">强化学习相关框架</td><td>VeRL</td><td>NPU、GPU</td><td rowspan="3">1、直接通过pip安装时：

pip show [name]

2、通过源码编译时，确认具体commit分支

git branch

输出所有分支，当前所在的分支将以*标记</td></tr>
<tr><td>vllm</td><td>NPU、GPU</td></tr>
<tr><td>vllm-ascend</td><td>NPU</td></tr>
</table>

2.1.2 模型与权重对齐

1、精度比对前需确保训练使用的模型权重版本一致，相同的模型可能有 Base、Instruct、Thinking 多个版本，初始权重不一致无法进行精度对齐。可以直接从 Huggingface/魔搭社区 重新拉取模型权重，或者通过 md5 等方式确保权重文件一致。

2、模型的配置文件也需要完全一致，如 tokenizer_config.json 中的 chat_template 可能被修改，去除了 think 功能导致模型推理输出不一致，造成精度差异。

- 权重文件对齐

模型中权重文件一般有多个，需要通过md5命令逐个对齐，确认权重文件完全一致

```
# 如果没有md5工具，先通过apt-get/yum安装

md5 model-00001-of-00002.safetensors
# 输出该文件的md5值
```

- 配置文件对齐

配置文件可以通过diff命令快速对齐，并直接输出两个文件的差异

```
diff -r dir1 dir2
# 输出两个文件夹dir1、dir2逐个文件对比的结果
```

2.1.3 数据集与输入预处理对齐

在强化学习过程中，由于多模型多阶段的训练模式，并且引入了推理采样，数据的误差在经过累积后会十分明显。数据对齐确保了模型在每一步训练和推理阶段接收到的输入数据有一致的分布和处理方式，是精度对齐的关键步骤。

- **数据集统一转换**

针对框架需要的输入方式，对数据集进行统一的处理。

VeRL框架接受parquet类型文件作为输入，对输入内容的规范和处理方式具体参照 `verl/docs/preparation/prepare_data.rst`

同时，verl也已经提供了gsm8k、math、HelloSwag、Full_hh_rlhf等数据集的预处理脚本，参考位置 `example/data_preprocess`

- **数据 shuffle**

1、关闭数据 shuffle

大模型训练中在数据加载时进行 shuffle，打乱数据顺序使其随机化，有助于提升模型训练的泛化效果和稳定性。在精度对齐过程中，可以通过关闭 shuffle 从而获得确定性和可复现性。

在verl训练启动脚本中添加如下配置：

```
data.shuffle=False \ # 需要设置为False，关闭shuffle
```

2、balance batch 策略对齐

在DP训练中，为了提高硬件利用率，使 device 间的显存负载均衡，会开启 `balance_batch` 选项，动态调整每个 dp rank 处理的数据批次顺序。通过对样本 token 数量的分析，重新分配数据，使同一批次内每张卡处理的有效 token 总数大致相等。该参数也会造成数据shuffle，在需要依赖批次内数据间特定关系的场景下需要关闭。在精度对齐过程中，balance batch策略也应该保持一致。

在verl训练启动脚本中添加如下配置：

```
trainer.balance_batch=True # 两边保持一致即可
```

- **超长数据处理**

对超长数据的处理方式（过滤或截断）影响输入数据的分布，同样需要保持一致。

```
# 以下三条配置，两边均保持一致即可
data.max_prompt_length=${max_prompt_length} \
data.filter_overlong_prompts=${filter_overlong_prompts} \
data.truncation='left' \
```

2.1.4 超参数对齐

代码、环境、模型和数据集完成对齐后，就可以开始进行强化学习训练了。强化学习中存在较多的参数，包括输入数据处理、rollout推理、并行切分策略相关的参数都会影响训练的输入输出和过程，从而导致精度出现偏差，因此在训练前需要进行参数对齐。

verl框架通过覆盖默认的参数配置进行强化学习训练，参数文件位置在`/verl/trainer/config` 目录下，ppo训练默认使用 `ppo_trainer.yaml`。这里列出一些关键参数作为参考，这些参数在精度对齐时要严格保持一致。

- **batch size相关参数**

VeRL	释义
data.max_prompt_length	最大输入提示词长度
data.max_response_length	最大生成响应长度
actor_rollout_ref. rollout.n	一条prompt推出的response数量
actor_rollout_ref.rollout.max_model_len	rollout阶段输出的 prompt+response 最长长度
data.train_batch_size	rollout阶段的batch size
actor_rollout_ref.actor.ppo_mini_batch_size	每次更新actor update使用的prompt数量

VeRL	释义
actor_rollout_ref.actor.ppo_micro_batch_size_per_gpu	在单个GPU上进行一次前向/反向的(Prompt, Response)对数量 这是实现梯度累计的核心参数，mini-batch会根据该参数再次切分为若干micro-batch
actor_rollout_ref.ref.log_prob_micro_batch_size_per_gpu	只进行前向计算的ref model 设置 micro-batch size
actor_rollout_ref.rollout.log_prob_micro_batch_size_per_gpu	对生成样本每次前向计算logp的(Prompt, Response)对数量
actor_rollout_ref.actor.use_dynamic_bsz actor_rollout_ref.ref.log_prob_use_dynamic_bsz actor_rollout_ref.rollout.log_prob_use_dynamic_bsz	是否启用dynamic batch size，动态打包样本，提高吞吐 开启时，系统会忽略基于样本数的 micro_batch_size_per_gpu 参数，转而使用基于 Token 数的 max_token_len_per_gpu 参数来构建 micro-batch
actor_rollout_ref.actor.ppo_max_token_len_per_gpu actor_rollout_ref.ref.log_prob_max_token_len_per_gpu actor_rollout_ref.rollout.log_prob_max_token_len_per_gpu	配合dynamic batch size使用，如果use_dynamic_bsz=True，需显式设置 作用在Actor类的 update_policy、compute_log_prob方法

- **并行策略相关参数**

VeRL	释义
actor_rollout_ref.rollout.tensor_model_parallel_size	推理阶段的TP，表示用多少个GPU来共同运行一个Engine

VeRL	释义
	例如，TP_SIZE=4 表示将一个大模型的权重切成 4 份，由 4 个 GPU 协同完成推理
actor_rollout_ref.rollout.data_parallel_size	推理阶段的DP大小
actor_rollout_ref.rollout.rollout_config.pipeline_model_parallel_size	推理阶段的PP大小
actor_rollout_ref.actor.fsdp_config.fsdp_size	fsdp切分参数，设置为-1则相当于world size

- **算法相关参数**

VeRL	释义
actor_rollout_ref.actor.grad_clip	actor更新的梯度裁剪系数
actor_rollout_ref. actor.optim.lr	学习率
algorithm.adv_estimator=grpo	Advantages计算方式
actor_rollout_ref. actor.kl_loss_type	kl_loss计算方式，grpo选择low_var_kl
actor_rollout_ref. actor. kl_loss_coef	kl_loss系数
algorithm.filter_groups.enable=False	是否开启DAPO filter 开启 filter 算法，过滤掉这类差异度较小的样本（所有response的奖励值几乎相同） 会提高推理采样的随机性，精度对齐时需要关闭

2.2 固定随机性

2.2.1 verl固定随机性

若在上一章节排查完基本的环境、参数、代码等影响因素后问题还存在，需要进一步定位问题，在定位之前需要尽量保证问题现象可重复复现。下方内容介绍在VeRL强化学习训练过程中，如何固定随机性，让实验可重复，提升问题定位效率。

1、固定随机种子

使用精度工具 msprobe，可以快速固定网络中的随机种子

精度工具安装使用

```
pip uninstall mindstudio-probe
# git clone https://gitee.com/ascend/mstt.git
git clone https://gitcode.com/Ascend/mstt.git
export PYTHONPATH=/path/mstt/debug/accuracy_tools:$PYTHONPATH
cd mstt/debug/accuracy_tools
pip install .
```

使用方法：VeRL等使用Ray进行分布式训练的框架，由于ray集群在启动时会将环境变量作为快照固定下来，快照一经生成即只读，后续再修改不会同步到已运行的raylet，必须通过runtime_env显式覆盖。因此如果只在主函数入口添加确定性，无法传递到worker中。由于训练的前反向过程都在worker中进行，因此不能简单地在训练入口文件（如：pretrain.py）开头固定随机种子，**需要在worker中添加如下代码**。

```
from msprobe.pytorch import seed_all
seed_all(seed=1234, mode=True)
```

参数说明：seed表示传入的随机数种子，默认值为1234；mode表示是否开启确定性计算模式

代码位置：

1、使用fsdp后端：verl\workers\fsdp_workers.py

2、使用megatron后端：verl\workers\megatron_workers.py

在对应Worker文件的开头添加seed_all操作

```python
verl > workers > fsdp_workers.py > ...
        init_fn,
        layered_summon_lora_params,
        load_fsdp_model_to_gpu,
        load_fsdp_optimizer,
        offload_fsdp_model_to_cpu,
        offload_fsdp_optimizer,
        replace_lora_wrapper,
    )
    from verl.utils.import_utils import import_external_libs
    from verl.utils.memory_utils import aggressive_empty_cache
    from verl.utils.model import compute_position_id_with_mask, convert_weight_keys
    from verl.utils.profiler import DistProfiler, DistProfilerExtension, ProfilerConfig, log_gpu_memory
    from verl.utils.profiler.performance import reduce_timing, topk_reduce_ratio_min_max
    from verl.utils.py_functional import convert_to_regular_types
    from verl.utils.ray_utils import get_event_loop
    from verl.workers.config import FSDPCriticConfig, FSDPEngineConfig, HFModelConfig, RolloutConfig
    from verl.workers.config.optimizer import build_optimizer
    from verl.workers.rollout import get_rollout_class
    from verl.workers.sharding_manager.fsdp_ulysses import FSDPUlyssesShardingManager

    from msprobe.pytorch import seed_all
    seed_all(seed=1234, mode=True)
```

2、固定推理随机性

1）固定vllm随机种子

launch vLLM Engine Server时使用同一个seed，并且设置temperature、top-k、top-p等采样参数相等，确保推理的结果大致相同。

seed生效代码位置 `verl\workers\rollout\vllm_rollout\vllm_async_server.py`

在verl启动脚本中添加如下配置：

```
# 在verl启动脚本中对齐
# Algorithm
temperature=1.0
top_p=1.0
top_k=-1 # 0 for HF rollout, -1 for vLLM rollout

# verl启动参数
+actor_rollout_ref.rollout.seed=1234
```

2）确保 infer 的 dp_size 一致

verl框架中根据rollout阶段的并行参数会影响推理的结果，因此verl训练任务的以下参数应该保持一致：

```
actor_rollout_ref.rollout.tensor_model_parallel_size
actor_rollout_ref.rollout.data_parallel_size
actor_rollout_ref.rollout.rollout_config.pipeline_model_parallel_size
```

原因在于worker在推理阶段 _build_rollout() 方法中，会根据 dp_size 固定一次随机种子，确保所有 tp_rank 拥有相同的随机状态

```
# 3. init trainer and rollout random states
self.torch_random_states = get_torch_device().get_rng_state()
gen_dp_rank = rollout_device_mesh["dp"].get_local_rank()
get_torch_device().manual_seed(gen_dp_rank + 1000)  # make sure all tp ranks have the same
self.gen_random_states = get_torch_device().get_rng_state()
get_torch_device().set_rng_state(self.torch_random_states)
```

dp_size的计算方法

```
infer_tp = rollout_config.tensor_model_parallel_size * rollout_config.data_parallel_size
infer_pp = rollout_config.pipeline_model_parallel_size
infer_world_size = infer_tp * infer_pp
dp =self.world_size // infer_world_size
```

3）框架调度随机性固定

vllm支持多种调度模式，如mp后端，ray后端，external_launcher后端，uni后端，其中mp和ray是属于多进程模式，exteral_launcher后端和uni后端是属于单进程模式。多进程相关的调度会加剧多请求处理时的batch变化，从而造成和上文中KV cacge驱逐类似的影响，为了规避多进程带来的输出内容差异，NPU和GPU在精度单推对齐时可以使能以下环境变量转为单进程调度：

```
export VLLM_ENABLE_V1_MULTIPROCESSING=0
```

2.2.2 NPU确定性配置（可选）

使用场景：如果配置已经完成对齐并且随机数已固定的情况下，仍然无法固定随机性，可以考虑添加以下的环境变量来使能确定性实现。

使用msprobe工具固定随机种子的过程中，还会自动设置相关环境变量【参考文档1】，进行确定性使能，因此一般使用msprobe seed_all() 即可。如确定性仍无法固定住，可以查看【参考文档3】中的实践进行尝试。

- 通信确定性

注意：LCCL_DETERMINISTIC 开关仅涉及LCCL确定性AllReduce是否开启（影响的Operation包括AllReduceOperation、RediuceScatterOperation、LinerParallelOperation），且开启功能仅在单机场景生效。该环境变量将在未来版本日落。

```
export HCCL_DETERMINISTIC=true
export LCCL_DETERMINISTIC=1
```

- 算子确定性

```
torch.use_deterministic_algorithms(True)
export CLOSE_MATMUL_K_SHIFT=1  # 关闭K轴错峰优化（补充使用场景，matmul v2单独的优化）
export ATB_MATMUL_SHUFFLE_K_ENABLE=0 # 关闭Shuffle-K使能
export ATB_LLM_LCOC_ENABLE=0
```

参考文档

1. msprobe_seed_all()源码

2. 安装后自动配置环境变量-CANN商用版8.3.RC1-昇腾社区

3. Align the Inference results of the verl and vLLM frameworks on Ascend devices(zh) — verl documentation
、

4. vLLM开发知识库-vllm推理精度对齐方案

2.2.3 快速配置方法（可选）

这里提供一个seed_all()方法实现，通过使用这个seed_all能快速完成固定随机性和确定性配置，使用方法同2.2.1 verl固定随机性，将seed_all函数在Worker类中调用一次即可（不需要导入msprobe）。

verl > workers > fsdp_workers.py > ...

```python
)
from verl.utils.import_utils import import_external_libs
from verl.utils.memory_utils import aggressive_empty_cache
from verl.utils.model import compute_position_id_with_mask, convert_weight_keys
from verl.utils.profiler import DistProfiler, DistProfilerExtension, ProfilerConfig, log_gpu_memory
from verl.utils.profiler.performance import reduce_timing, topk_reduce_ratio_min_max
from verl.utils.py_functional import convert_to_regular_types
from verl.utils.ray_utils import get_event_loop
from verl.workers.config import FSDPCriticConfig, FSDPEngineConfig, HFModelConfig, RolloutConfig
from verl.workers.config.optimizer import build_optimizer
from verl.workers.rollout import get_rollout_class
from verl.workers.sharding_manager.fsdp_ulysses import FSDPUlyssesShardingManager

import torch_npu
import numpy as np
import random
from torch_npu.contrib import transfer_to_npu

def seed_all(seed=1234):
    random.seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)
    os.environ['HCCL_DETERMINISTIC'] = str(True)
    os.environ['LCCL_DETERMINISTIC'] = str(1)
    os.environ['CLOSE_MATMUL_K_SHIFT'] = str(1)
    os.environ['ATB_MATMUL_SHUFFLE_K_ENABLE'] = str(0)
    os.environ['ATB_LLM_LCOC_ENABLE'] = str(0)
    os.environ["VLLM_ENABLE_V1_MULTIPROCESSING"] = str(0)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.use_deterministic_algorithms(True)
    torch_npu.npu.manual_seed(seed)
    torch_npu.npu.manual_seed_all(seed)

seed_all()
```


```python
import torch_npu
import numpy as np
import random
from torch_npu.contrib import transfer_to_npu

def seed_all(seed=1234):
    random.seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)
    os.environ['HCCL_DETERMINISTIC'] = str(True)
    os.environ['LCCL_DETERMINISTIC'] = str(1)
    os.environ['CLOSE_MATMUL_K_SHIFT'] = str(1)
    os.environ['ATB_MATMUL_SHUFFLE_K_ENABLE'] = str(0)
    os.environ['ATB_LLM_LCOC_ENABLE'] = str(0)
    os.environ["VLLM_ENABLE_V1_MULTIPROCESSING"] = str(0)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.use_deterministic_algorithms(True)
    torch_npu.npu.manual_seed(seed)
```

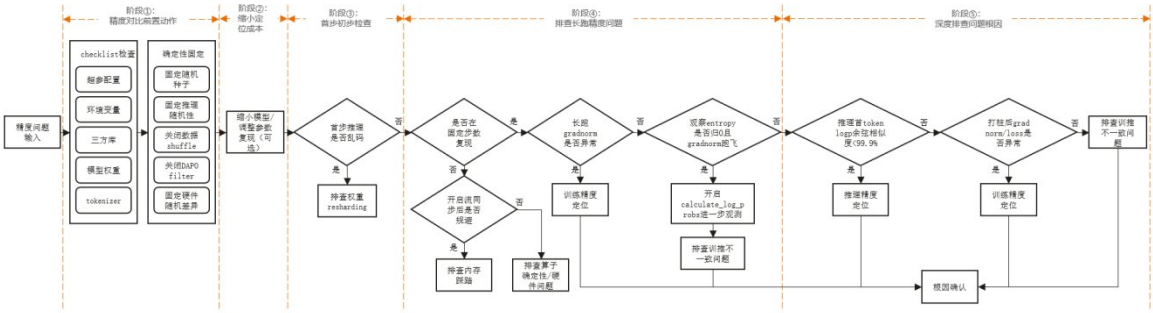
```
    torch_npu.npu.manual_seed_all(seed)

seed_all()
```

3. 精度问题定位

3.1 精度问题定位流程

强化学习整体精度定位流程如图所示。



本章主要介绍强化学习精度问题定位的整体流程和具体步骤，旨在帮助用户快速排查可能影响精度的因素，定位引发精度问题的根因，并举一反三迁移到其他更加具体和复杂的问题场景。整体流程主要分为五个阶段，通过结合强化学习中的关键指标，帮助用户快速定界精度问题由**训练阶段/推理阶段/训推一致性**问题引入，并参考具体定位流程确认根因。

强化学习精度定位流程主要分为以下五个阶段：

1.**阶段①：精度对比前置动作**，分为checklist检查和确定性固定两部分，主要目的为排除超参、环境变量、三方库版本、模型配置文件等因素对精度结果的干扰，并保证实验结果的可重复性；对于DAPO算法，建议设置algorithm.filter_groups.enable=False关闭DAPO filter

2.**阶段②：缩小定位成本**，包括缩小模型、调整gbs等手段，建议根据实际场景和现场调试成本自行判断

3.**阶段③：首步初步检查**，直观判断首步推理结果是否出现推理乱码现象、推理结果与GPU差异大或者结果重复等；如存在以上问题，优先排查权重resharding问题

4.**阶段④：排查长跑精度问题**，固定确定性后，长跑精度问题按是否稳定复现分为两类：1）若为不稳定复现场景，优先排查开启流同步后能否规避，若可规避按内存踩踏问题进行定位；若仍无法规避考虑进一步排查算子确定性问题、硬件问题、模型及框架层面等，详见3.4节；2）若为稳定复现场景，观察长跑gradnorm是否异常，若存在尖刺、nan值、与GPU相差较大（超过一个数量级）则优先按训练精度问题排查，手段及排查思路详见

3.3.3节；若满足entropy归零且gradnorm跑飞，则优先按训推不一致问题排查，可开启calculate_log_probs参数进一步观测

5.**阶段⑤：深度排查问题根因**：若上述的观测指标均无异常，可计算NPU与GPU推理首token logp余弦相似度，若该值<99.9%，则优先按推理精度定位，详见3.3.2节；若首token推理无异常，则打桩加载GPU的推理结果，观察gradnorm/pg_loss与GPU的误差，当误差>1%时需要按训练精度问题定位；若仍无异常，考虑训推一致性情况。

3.2 缩小精度问题定位成本

在正式定位训练或推理的精度问题之前，如果精度问题发生时所占用的资源较多，或复现时间较长，会导致复现和定位精度问题成本过高。可以缩小精度问题定位成本，节省定位所需的资源和时间成本。

1. 缩小配置参数复现

在尝试复现精度问题时，可以先尝试缩小以下配置参数进行复现

全局批次大小	train_prompt_bsz
生成批次大小	gen_prompt_bsz
推理样本个数	n_resp_per_prompt
最大prompt长度	max_prompt_length
最大response长度	max_response_length
推理实例个数	通过修改总卡数实现，推理实例个数=NNODES // gen_dp // gen_tp
推理DP	gen_dp
训练DP	通过修改总卡数实现，训练DP = NNODES // train_tp // train_pp // train_cp

需注意的是，缩小训练DP会增加单卡显存占用，存在OOM风险：在Megatron中，由于通常启用分布式优化器，缩小DP会显著增加优化器显存占用；在FSDP或DeepSpeed中，减少卡数会导致模型分片数下降，单卡负载上升。因此，仅当原训练配置显存余量充足时，才建议尝试减小DP。

2. 缩小模型规模复现

以qwen_moe模型为例，qwen3-30B与qwen3-235B的实现逻辑相同，若qwen3-235B出现精度问题，可尝试使用qwen3-30B模型复现，若能复现就可以在qwen3-30B的基础上定位。

3. 推理结果打桩

如果能够明确是训练问题，如推理精度已经对齐，或在精度已经对齐的实验基础上，调整训练配置后出现精度问题，可以打桩推理结果，以跳过推理流程，加快问题复现速度，缩小复现时间成本。

3.3 可稳定复现问题定位

3.3.1 打桩方法

进行精度问题定位时，**打桩**是一种比较有效且能够快速复现问题的方法，本节主要介绍打桩的基本概念及方法，便于后续使用该方式进行问题定位。

打桩概念：打桩即保留当前阶段的输入输出数据，便于从结果上对比分析，如下图所示，将batch的数据dump下来即为打桩操作。我们在进行精度问题排查时，需要进行打桩辅助问题定位。常见的打桩方式是直接将rollout阶段的数据直接dump下来，保存为pkl或jsonl。verl中的skip rollout工具支持该功能（具体见3.2.1.1 skip rollout工具），在进行问题定位时，可以在GPU机器前几步的推理结果进行打桩，在NPU上直接基于GPU的rollout结果进行训练，观察前几步loss及grad norm曲线差异是否较大。

```
# all the tp ranks should contain the same data here. data in all ranks are valid
batch = TensorDict(
    {
        "prompts": idx,
        "responses": response,
        "input_ids": seq,  # here input_ids become the whole sentences
        "attention_mask": attention_mask,
        "position_ids": position_ids,
    },
    batch_size=batch_size,
)
if self.config.calculate_log_probs:
    # we will recompute old log prob with actor
    batch["rollout_log_probs"] = rollout_log_probs

return DataProto(batch=batch, non_tensor_batch=non_tensor_batch)
```

3.3.1.1 skip rollout工具打桩

PR链接：[rollout] feat: add rollout_skip to skip rollout by reusing previously generated sequences by wlf-darkmatter · Pull Request #2602 · volcengine/verl · GitHub

工具简介：

skip rollout是通过缓存并复用之前生成的序列来加快强化学习训练中的rollout过程的工具，其具体适用场景有两个：

- **需重复运行相同配置实验时：**当研究者或开发者需要基于**完全一致的实验参数**（如环境设定、算法超参数、初始状态等）多次开展训练实验（例如验证结果稳定性、对比不同优化模块效果）时，无需每次都重新让智能体与环境交互生成全新序列 —— 可直接复用之前缓存的序列，避免重复劳动。
- **希望节省时间、规避冗余序列生成以接近最优策略时：**在强化学习训练中，接近“最优策略”的过程中，部分阶段生成的序列具有一定复用价值。若每次训练都从头生成所有序列，会产生大量冗余计算；而通过 RolloutSkip 复用已有序列，能减少不必要的交互开销，快速推进训练进程，更快向最优策略收敛。
  应用于我们的问题定界场景，skip rollout可以快速验证训练阶段问题是否能够稳定复现，另外可以基于该工具直接打桩rollout阶段的数据进行后续对比分析。

使用方法：

skip rollout目前已合入verl仓上，RayDAPOTrainer()以及RayPPOTrainer()均已进行适配，只需要在脚本中添加如下两行配置即可：

```
actor_rollout_ref.rollout.skip_rollout=True \
actor_rollout_ref.rollout.skip_dump_dir="/tmp/rollout_dump" \
```

skip_dump_dir配置说明：

1. 目录的核心作用与基础要求

- skip_dump_dir是专门用于存放“缓存序列”（训练中可复用的数据序列，减少重复计算）的文件夹。

- 必须满足两个前提：① 该目录对训练进程有写入权限（能存数据）；② 训练进程能访问到该目录（能读数据），否则缓存无法正常使用。

2. 关键注意事项：禁止使用相对路径

- 原因与 ray（分布式计算框架，常用于大规模训练）的机制有关：ray 会默认将数据存放在 /tmp/ray/session_<session_id>/这类临时路径下。

- 如果 skip_dump_dir 用相对路径（如 ./cache），训练的worker会在 ray 的临时路径下寻找该相对路径，而非用户预期的位置，最终导致找不到缓存目录。因此必须用绝对路径（如 /home/user/train_cache）

3. 缓存数据的路径命名规则

- 缓存数据会按固定格式生成文件夹名称，格式为：{experiment_name}_{project_name}_TrainGBS{train_gbs}__InferGBS{gen_gbs}__N{n}。

- 其中 experiment_name（实验名）、project_name（项目名）、train_gbs（训练批次大小）、gen_gbs（推理批次大小）、n（推理回答输出数量）这5个参数，只要任意一个发生变化，就会生成一个新的缓存目录，避免不同配置的缓存数据混淆。

skip_rollout v2版（尚未合入，修复打桩功能并支持连续打桩）： [rollout, tool] feat,fix: Rollout Skip v2 update with dump_steps > 1 supporting by wlf-darkmatter · Pull Request #3283 · volcengine/verl

3.3.2 推理精度对齐

3.3.2.1 影响推理精度的因素

影响因素/参数		**说明**
算子随机性影响		计算算子和通信算子的确定性计算固定方式见 2.2 节
参数	reward_model.overlong_buffer.enablereward_model.overlong_buffer.len reward_model.overlong_buffer.penalty_factor	超长惩罚相关参数，超出限定长度会对reward施加惩罚，影响reward对齐
	algorithm.filter_groups.enable algorithm.filter_groups.max_num_gen_batches	动态采样相关参数，会筛选掉reward满分和0分的response，如果筛选后数量未达到train_batch_size大小，会进行第二轮采样，达到max_num_gen_batches轮未采够后会异常退出
	data.gen_batch_size actor_rollout_ref.rollout.n	推理生成batch相关参数，需对齐否则会出现精度差异，缩小\gen_prompt_bsz可能会出现精度下降
	actor_rollout_ref.actor.fsdp_config.fsdp_size	指定fsdp训练后端切分卡数，不同切分间精度曲线存在差异

	actor_rollout_ref.actor.entropy_coeff	决定entropy是否对reward产生影响
环境变量	VLLM_ASCEND_ENABLE_NZ="0" / "1"	开启可能导致训推一致性差异扩大，目前默认开启，**RL场景推荐关闭**
	VLLM_ASCEND_ENABLE_TOPK_TOPP_OPTIMIZATION="0" / "1"	vllm-ascend重写了vllm的Sampler类，引入了topp topk融合算子来优化性能，在使能topp、topk场景下，可以使能以下环境变量Disable该融合算子来排查该部分可能的精度问题

3.3.2.2 推理结果对齐

1.（可选）缩小推理精度对齐成本

在dump推理数据之前，若复现推理精度问题占用的资源较多，可以先尝试缩小推理精度问题复现成本，减少复现的规模，减少需要dump和对比的数据。

在多batch、长序列的场景下，可以通过发送单batch请求，减少序列长度尝试复现。同时可以排查问题是否是因为chunk prefill特性，或者多batch下的算子问题导致。

2.vllm固定精度的方法

见 2.2 节固定随机性

3. 推理数据dump

在推理或训练过程中，模型可能出现输出偏离预期、生成异常、甚至产生 NaN/Inf 等数值不稳定问题。要定位根因，需要对模型执行路径进行精细化监控，采集中间特征、权重、激活值以及各关键层的输入输出，并记录提示词、张量 dtype、硬件配置等上下文信息。通过捕获这些核心张量及元数据，我们可以系统性地追踪精度退化或数值错误的来源。那我们需要通过msprobe工具进行数据dump。

msprobe 支持三种精度级别：

- **L0**：在nn.Module级别保存tensor，并生成 construct.json 以便可视化工具还原网络结构，需要传入模型或子模块句柄。
- **L1**：仅采集算子级统计信息，适合轻量排查。
- **mix**：同时获取结构信息与算子统计，适用于既要构图又要进行数值对比的场景。

安装：

```
pip install mindstudio-probe==8.3.0
pip install tb_graph_ascend
```

在vllm-ascend里使能msprobe

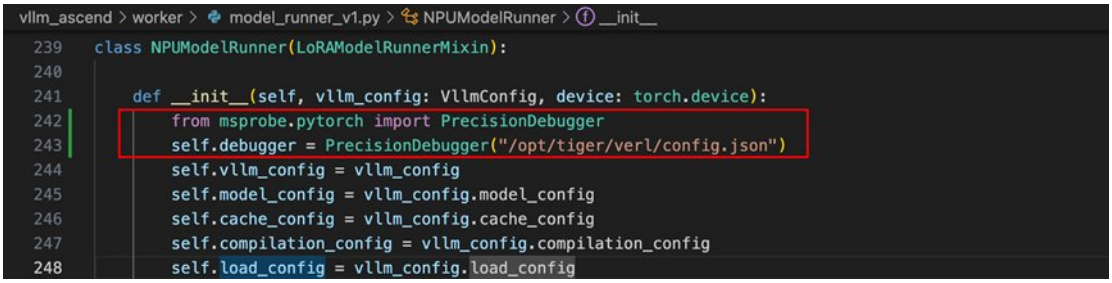
① 提前准备好config.json配置文件，默认采集所有rank，所有step；可以在list里面传入具体算子名字，采集特定算子；data_mode可以控制前向+反向

```
{
    "task": "tensor",
    "dump_path": "/opt/tiger/verl/dump_data/dummy_l0",
    "rank": [],
    "step": [0],
    "level": "L0",

    "statistics": {
        "scope": [],
        "list": [],
        "data_mode": ["all"],
        "summary_mode": "statistics"
    }
}
```

② ModelRunner初始化这里添加

```
    from msprobe.pytorch import PrecisionDebugger
    self.debugger = PrecisionDebugger("/opt/tiger/verl/config.json")
```



③ execute_model入口处添加：

```
vllm_ascend > worker > model_runner_v1.py > NPUModelRunner > execute_model
        return moe_comm_type

    @torch.inference_mode()
    def execute_model(
        self,
        scheduler_output: "SchedulerOutput",
        intermediate_tensors: Optional[IntermediateTensors] = None,
    ) -> Union[ModelRunnerOutput, AsyncModelRunnerOutput, IntermediateTensors]:
        self.debugger.start(self.model) # level=0
        # self.debugger.start() # level=1
        with ProfileExecuteDuration().capture_async("prepare input"):
            self._update_states(scheduler_output)
            if not scheduler_output.total_num_scheduled_tokens:
                if not has_kv_transfer_group():
```

④ execute_model函数return之前添加:

```
vllm_ascend > worker > model_runner_v1.py > NPUModelRunner > execute_model
                get_kv_transfer_group().clear_connector_metadata()

        extra_args = ({"kv_connector_output": kv_connector_output})

        model_runner_output = ModelRunnerOutput(
            req_ids=req_ids_output_copy,
            req_id_to_index=req_id_to_index_output_copy,
            sampled_token_ids=valid_sampled_token_ids,
            logprobs=logprobs_lists,
            prompt_logprobs_dict=prompt_logprobs_dict,
            pooler_output=[],
            **extra_args,
        )
        self.debugger.stop()
        self.debugger.step()

        durations = ProfileExecuteDuration().pop_captured_sync()
        if durations:
```

4. 首 Token logits 对比方法

精度强对齐场景下，会对 rollout 首 token 的 logits 进行强对齐比对，在此给出首 token 的 logits 抓取位置:

vllm / vllm / v1 / sample / sampler.py

Code Blame 285 lines (246 loc) · 11.1 KB ·

```python
class Sampler(nn.Module):

    def forward(
        self,
        logits: torch.Tensor,
        sampling_metadata: SamplingMetadata,
    ) -> SamplerOutput:
        # NOTE(woosuk): Use the original logits (before any penalties or
        # temperature scaling) for the top-k logprobs.
        # This is different from the V0 sampler, which uses the logits that
        # is used for sampling (after penalties and temperature scaling).
        num_logprobs = sampling_metadata.max_num_logprobs
        if num_logprobs is not None:
            if self.logprobs_mode == "raw_logprobs":
                raw_logprobs = self.compute_logprobs(logits)
            elif self.logprobs_mode == "raw_logits":
                raw_logprobs = logits.clone()

        # Use float32 for the logits.
        logits = logits.to(torch.float32)
        # Apply allowed token ids.
        logits = self.apply_allowed_token_ids(logits, sampling_metadata)
```

3.3.2.3 resharding校验

在推理正式开始前，vllm会进行“dummy run”，通过来推理一个token来评估推理时的显存占用，进而分配显存，可以在vllm的LLM初始化时指定参数load_format来指定dummy run的权重是随机初始化的（dummy）还是真实权重（safetensors）。在verl中，通过参数 actor_rollout_ref.rollout.load_format 指定该参数。

当出现**推理乱码现象**时，如果引擎初始化方式为load_format=dummy，则sharding高概率存在问题，即使换成了safetensors后吐字正常，sharding也是存在问题的，需要对比前向。

轻量级对比:

情形	推理 load_format=dummy	推理load_format=safetensors	初诊结果
组合1	❌乱码	✅吐字正常	sharding有问题
组合2	❌乱码	❌乱码	权重有问题
组合3	✅吐字正常	✅吐字正常	这里没问题，其它问题

组合4	✅吐字正常	❌乱码	不可能

细致级对比:

步骤	操作1	操作2	对比第一步推理前向	结论
步骤1	推理 load_format=safetensors	推理 load_format=dummy	对齐✅	其它问题
			没对齐❌	sharding有问题 ①实际参与运算的张量与来源权重**地址不一致**。 ②更新权重后处理时**未覆盖**部分实际参与运算的张量
步骤2（请先执行步骤①）	推理 load_format=safetensors	**注释掉sharding**， 推理 load_format=safetensors	对齐✅	OK
			没对齐❌	训练权重加载有问题 ①（use_dist）hf->megatron权重转换有问题 ②（use_mbridge）mbridge适配有问题

下列代码是一个落盘权重的示例，通过在 verl/workers/rollout/vllm_rollout/vllm_rollout_spmd.py 调用 dump_CustomDeepseekV2DecoderLayer()方法， 对相关的权重进行落盘操作；落盘后通过调用 load_CustomDeepseekV2DecoderLayer()方法读取来比对。

```
class CompareWeight:

    @staticmethod
    def dump_CustomDeepseekV2DecoderLayer(path_for_save: Path, layer):

        path_for_save = Path(path_for_save).joinpath(f"CustomDeepseekV2DecoderLayer_{layer.layer_idx}"
)
```

```
    path_for_save.mkdir(exist_ok=True, parents=True)

    dict_tensor = {
        **dict(layer.named_parameters()),
        **{
            "self_attn.mla_attn.impl.W_UV": layer.self_attn.mla_attn.impl.W_UV,
            "self_attn.mla_attn.impl.W_UK_T": layer.self_attn.mla_attn.impl.W_UK_T,
            "self_attn.mla_attn.impl.q_proj.weight": layer.self_attn.mla_attn.impl.q_proj.weight,
        }
    }
    for name, param in dict_tensor.items():
        torch.save(param.cpu(), path_for_save.joinpath(name + ".pt"))
    return

@staticmethod
def load_CustomDeepseekV2DecoderLayer(path_for_save: Path):
    dict_tensor = {}
    for name in path_for_save.glob("*"):
        if not name.name.endswith(".pt"):
            name.rename(name.parent / (name.name + ".pt"))

    for name in path_for_save.glob("*.pt"):
        dict_tensor[name.name] = torch.load(path_for_save.joinpath(name))
    return dict_tensor
```

第一次落盘：读取了 safetensors 的 vllm推理引擎的权重

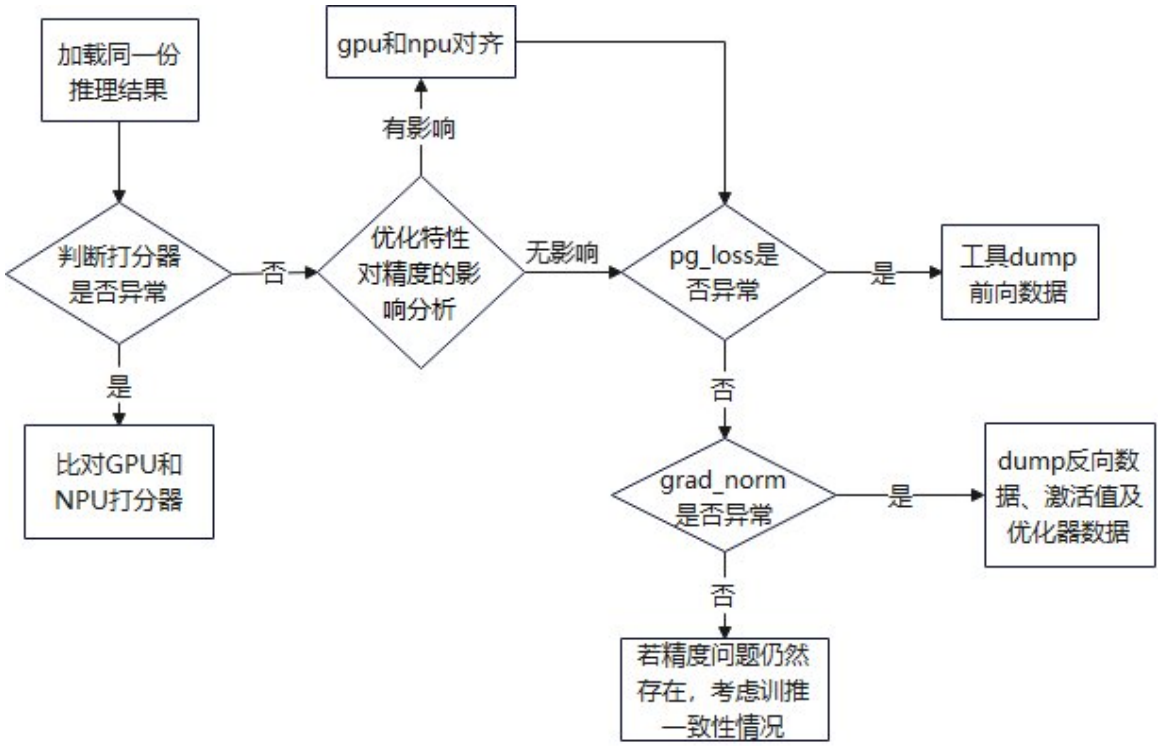
第二次落盘：第一次 sharding 后，vllm推理引擎的权重

对比两者权重即可判断sharding是否正确。

说明：在针对DeepSeek模型时，由于用到了MLA，推理引擎替换了attn的实现，还有额外的权重需要打开来对比。类似的其他模型如果有这类操作同样需要注意将其落盘下来对比。

3.3.3 训练精度对齐

相较常规大语言模型的预训练-微调范式，强化学习（RL）在训练阶段的技术实现上新增了诸多关键环节和核心模块，这是其与传统训练模式的核心差异所在。本章节将基于训练阶段为大家阐述强化学习场景下，打桩推理结果，保证训练阶段输入一致后，整体定位流程如下图所示：

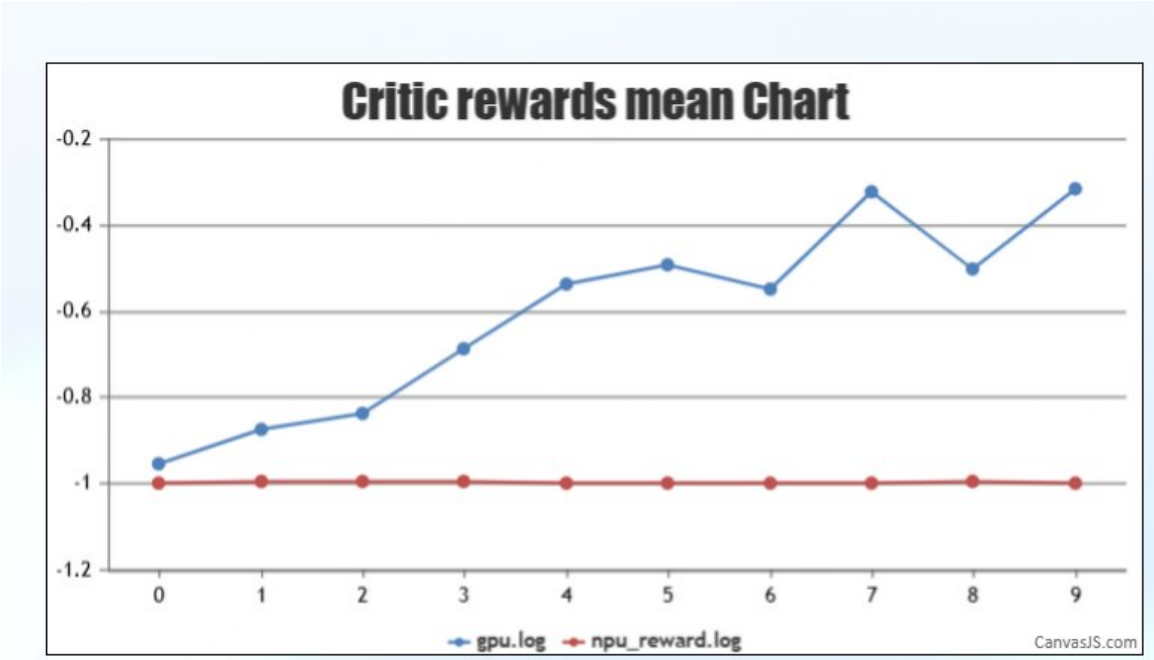


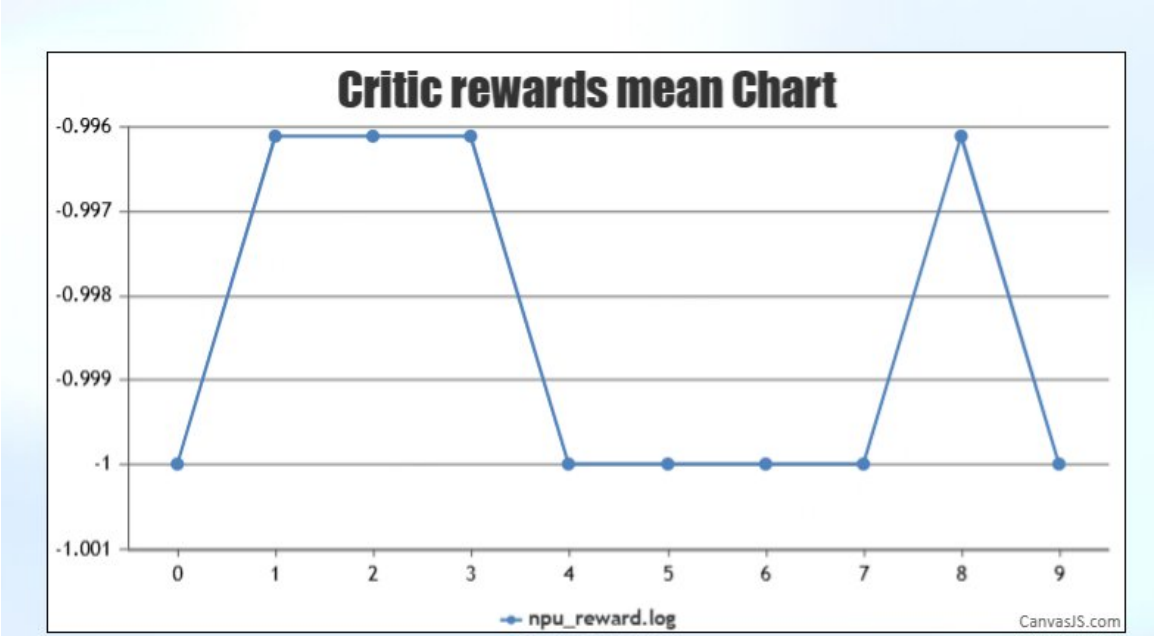
3.3.3.1 检查reward计算是否对齐

强化学习的学习依赖于Value Model和Reward Model（大部分热点算法已经没有了Value Model），当观测到NPU和GPU的Reward曲线不一致时，需要针对该部分做进一步实验观测。

1) 如何评估不一致:

A: GPU有得分而NPU没有得分，是明显的不一致





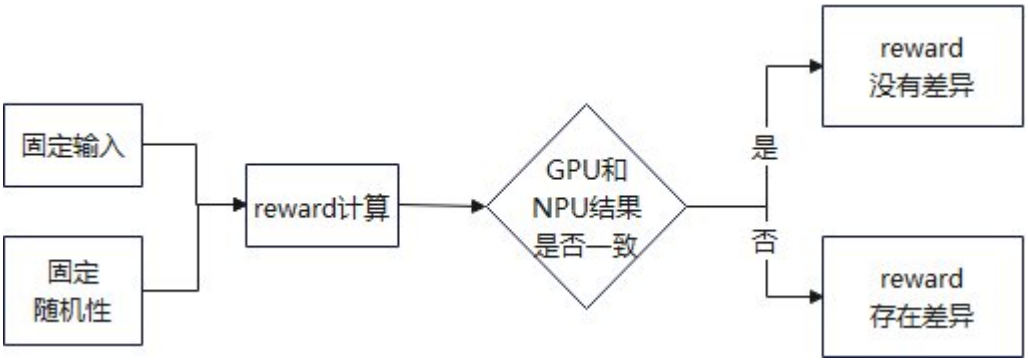
B: GPU有得分，NPU也有得分，但是两者差异较大（得分曲线几乎不交叉，存在明显的分层，分层的截距保持在**0.05**以上）

C: GPU得分和NPU得分曲线趋势不一致，两者上升速度不一样**（存在问题的可能性较低，但仍不能排除打分器问题）**

2）打分器为什么容易出问题，打分器有不同的类别：

数据集类型	打分方式	对不齐原因
Math	规则打分	1. 打分相关代码实现不同
Code	沙箱打分	1. 沙箱运行逻辑有差异
通用	模型打分	1. 外部模型调用的api不一致； 2. 本地部署的打分模型有问题； 3. 没有用相同的模型打分； 4. 打分模型温度过高（打分模型输出不固定）

3）如何判断打分器是否存在问题



固定输入（类似 3.3.1 打桩方法）和固定随机性（见 2.2固定随机性），对比NPU和GPU的输出是否一致。

- 确定对应的compute_score打分器
- 在该函数内增加breakpoint()，使用ray distributed debugger进行调试，保存compute_score的输入solution_str和gts
- 针对相同的solution_str和gts输入值，GPU和NPU分别单用例测试查看输出结果，结果一致则reward没有问题；不一致则需比对GPU和NPU的reward代码

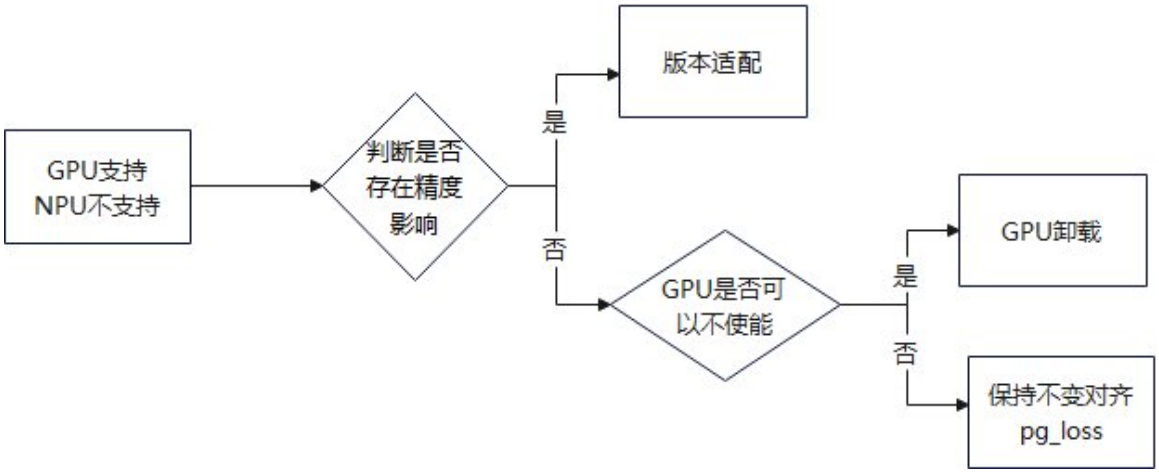
3.3.3.2 优化特性对精度影响分析及操作

随着生态版本快速演进，NPU和GPU相关特性支持度不一致，在精度比对时，针对所涉及特性需要定点详细分析

场景	GPU	NPU

情形1	不支持	支持
情形2	支持	不支持
情形3	支持	支持

- 针对情形1，NPU不使能相关特性，保持GPU和NPU一致
- 针对情形2，判断该特性是否对精度存在影响



- 针对情形3，保持GPU和NPU一致

  有些对精度产生影响，给一个历史经验参考

类型		描述
切分策略	Megatron的切分和FSDP的切分	切分策略一般不应该影响精度，但需跟GPU保持一致，否则可能引入累积误差或走入不同代码分支中等问题
通信优化	moe_shared_expert_overlap	在Megatron模型中，overlap类参数存在较高风险，可先关闭该类参数。
融合算子	FA	融合算子功能复杂全面，精度比对阶段可选择GPU和NPU都关闭进行对齐
	Rms_Norm	
	moe_premute_fusion	
混合精度	mixed_precision	大模型训练一般是BF16/FP32混合精度，需关注是否有特意修改相关参数，需保持对齐
	mp_policy	
参数同步	sync_module_states	初始化模型时，是否从rank0进行广播，用于保证参数一致性

Megatron及FSDP详细特性说明及影响可见文档：
https://wiki.huawei.com/domains/4600/wiki/241354/WIKI202511279181854

3.3.3.3 pg_loss差异定位思路

1) 问题描述

在打桩输入相同推理结果，训练配置保持一致，并且固定随机性的情况下，GPU和NPU的第一步或前几步的pg_loss就已经出现差异**（差异大于1%）**

2) 排查思路

按如下步骤进行定位：

1. 使用精度采集工具，采集最先出现差异的步数：

- 若模型比较大，层数比较多，可判断减少层数是否复现问题，若问题复现可减少层数再进入下一步
  注意：Moe模型场景下，一般前几层为Dense层，模型规模缩小时需最少保留一层Moe层，再减少Dense层
- 若第1步Loss（对应训练为第0步）不对齐，则使用工具采集第一步NPU与标杆的统计量数据，并排查其前向的计算结果。

2. 精度数据采集位置如下，其配置按照精度采集即可

verl-main > verl > workers > actor > megatron_actor.py

```python
class MegatronPPOActor(BasePPOActor):
    def update_policy(self, dataloader: Iterable[DataProto]) -> dict:

        ###debuger init ###
        from msprobe.pytorch import PrecisionDebugger, seed_all
        seed_all(mode=True) # deterministic
        debugger = PrecisionDebugger(dump_path=dump_path, config_path="path/../msprobe.json", level="L0")
        ###
        metrics = {}
        if self.use_torch_profiler and self.prof and self.prof.enable:
            self.prof.start()
        for data in dataloader:
            if self.config.router_replay.mode in ["R2", "R3"]:
                RouterReplay.set_global_router_replay_action(RouterReplayAction.REPLAY_FORWARD)
            self.actor_optimizer.zero_grad()
            # use use_contiguous_buffers_in_local_ddp and no overlap_dp_param_comm
            for chunk in self.actor_module:
                # if use distributed optimizer, zero grad buffer will be handled by optimizer
                chunk.zero_grad_buffer()

            calculate_entropy = self.config.entropy_coeff != 0
            if data.meta_info.get("micro_batch_size", None) is not None:
                micro_batch_size = data.meta_info["micro_batch_size"]
            else:
                micro_batch_size = self.config.ppo_micro_batch_size_per_gpu
            max_token_len = None
            if self.config.use_dynamic_bsz:
                max_token_len = self.config.ppo_max_token_len_per_gpu * self.config.megatron.context_parallel_size
            ####debuger start ###
            debugger.start(model=self.actor_module)
            ###
            metric_micro_batch = self.forward_backward_batch(
                data,
                calculate_entropy=calculate_entropy,
                use_dynamic_bsz=self.config.use_dynamic_bsz,
                micro_batch_size=micro_batch_size,
                max_token_len=max_token_len,
                mini_batch_size=self.config.ppo_mini_batch_size,
            )
            ####debuger stop ###
            debugger.stop()
            debugger.step()  # step +1
            ###
```

3. 使用分析工具进行比对，寻找可疑算子：

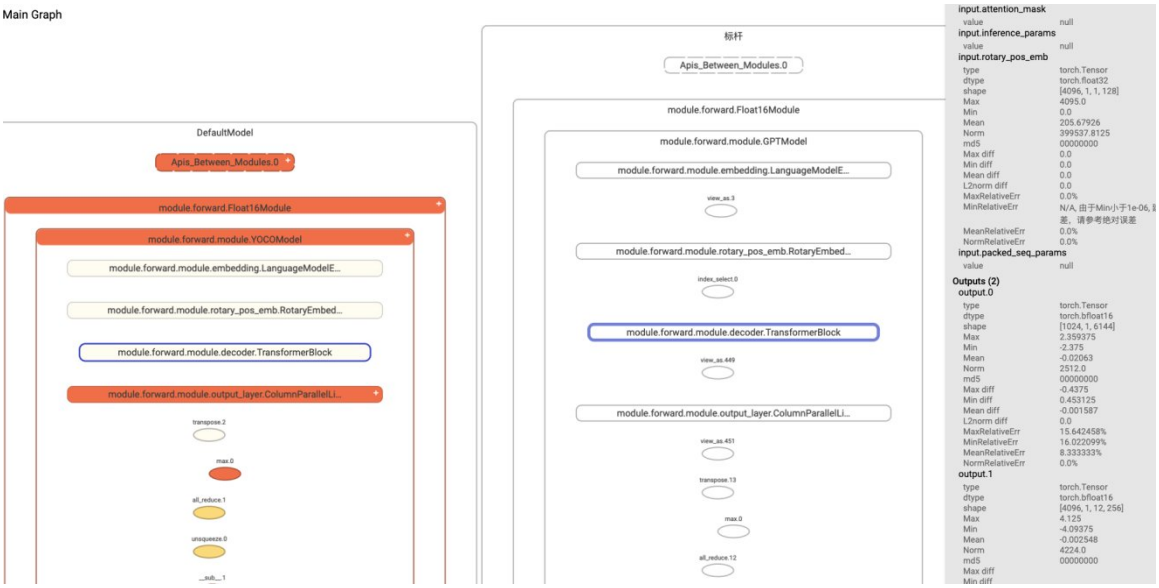
- 使用分级可视化工具做画图比对，结合精度筛选功能（颜色深到浅）+首个精度出现差异的节点来进行排查分析。

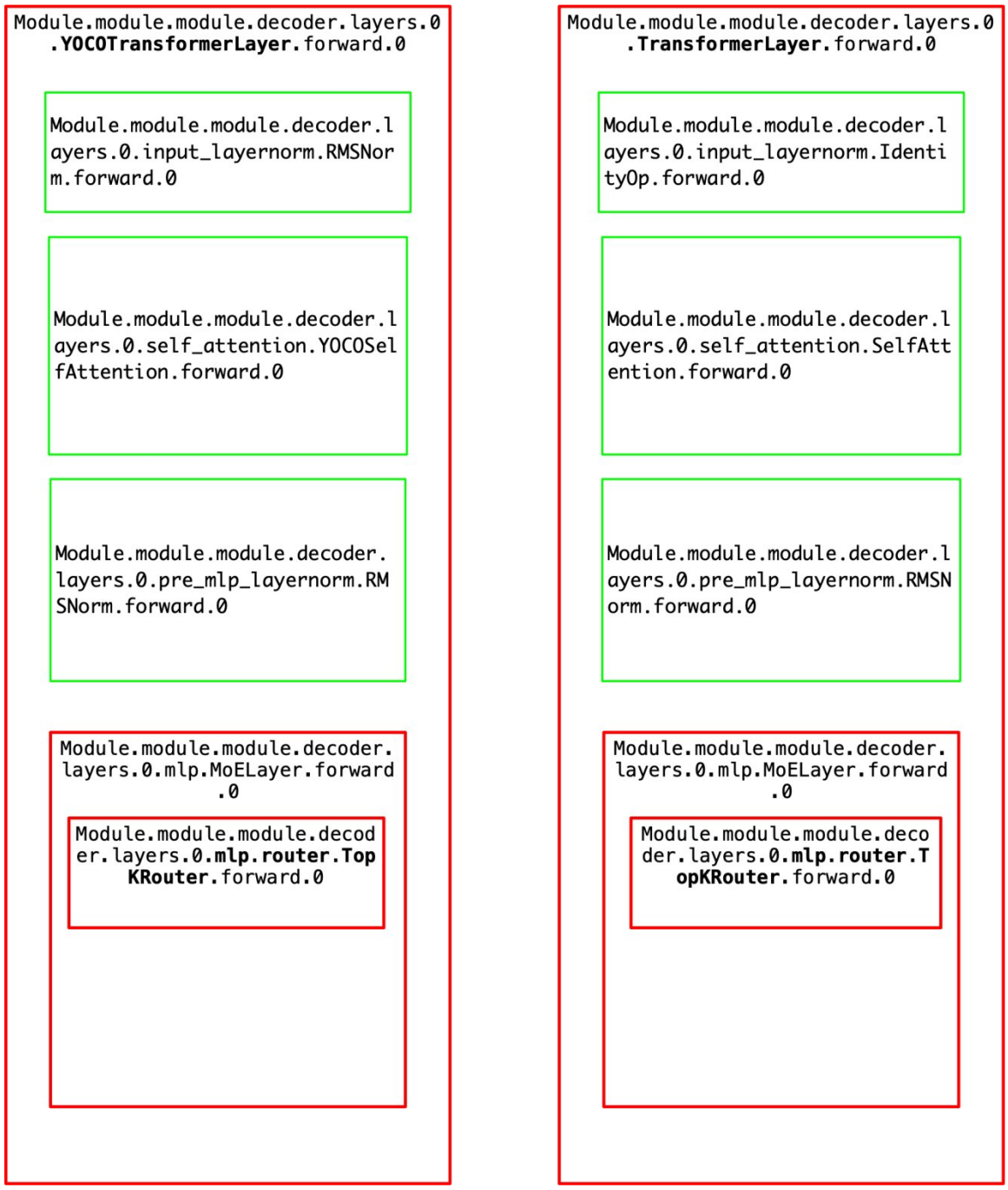
- 也可使用精度比对工具做表格比对，通过分析输入输出精度差异来进行排查分析。

4. 若没有定界到可疑算子，可以使用精度预检工具进行辅助排查。

示例：通过精度可视化对比工具定位到存在问题的模块

Main Graph


DefaultModel
Apis_Between_Modules.0
module.forward.Float16Module
module.forward.module.YOCOModel
module.forward.module.embedding.LanguageModelE...
module.forward.module.rotary_pos_emb.RotaryEmbed...
module.forward.module.decoder.TransformerBlock
module.forward.module.output_layer.ColumnParallelLi...
transpose.2
max.0
all_reduce.1
unsqueeze.0
__sub__.1
标杆
Apis_Between_Modules.0
module.forward.Float16Module
module.forward.module.GPTModel
module.forward.module.embedding.LanguageModelE...
view_as.3
module.forward.module.rotary_pos_emb.RotaryEmbed...
index_select.0
module.forward.module.decoder.TransformerBlock
view_as.449
module.forward.module.output_layer.ColumnParallelLi...
view_as.451
transpose.13
max.0
all_reduce.12
input.attention_mask
value null
input.inference_params
value null
input.rotary_pos_emb
type torch.Tensor
dtype torch.float32
shape [4096, 1, 1, 128]
Max 4095.0
Min 0.0
Mean 205.67926
Norm 399537.8125
md5 00000000
Max diff 0.0
Min diff 0.0
Mean diff 0.0
L2norm diff 0.0
MaxRelativeErr 0.0%
MinRelativeErr N/A, 由于Min小于1e-06, 相
差，请参考绝对误差
MeanRelativeErr 0.0%
NormRelativeErr 0.0%
input.packed_seq_params
value null
Outputs (2)
output.0
type torch.Tensor
dtype torch.bfloat16
shape [1024, 1, 6144]
Max 2.359375
Min -2.375
Mean -0.02063
Norm 2512.0
md5 00000000
Max diff -0.4375
Min diff 0.453125
Mean diff -0.001587
L2norm diff 0.0
MaxRelativeErr 15.642458%
MinRelativeErr 16.022099%
MeanRelativeErr 8.333333%
NormRelativeErr 0.0%
output.1
type torch.Tensor
dtype torch.bfloat16
shape [4096, 1, 12, 256]
Max 4.125
Min -4.09375
Mean -0.002548
Norm 4224.0
md5 00000000
Max diff
Min diff



3.3.3.4 grad_norm差异定位分析

1) 问题描述

在打桩输入相同推理结果，训练配置保持一致，并且固定随机性的情况下，GPU和NPU的第一步的 pg_los对齐，但Grad_norm对不齐（差异大于1%）

2) 排查思路

1.使用精度采集工具，采集精度数据

- 按照精度数据采集方法，采集NPU和标杆的反向数据，通过可视化工具比对计算结果
- 若反向数据无明显怀疑对象，使用工具Monitor训练状态监控工具采集NPU与标杆的Monitor训练状态数据，获取相关激活值、权重梯度、优化器状态和通信算子的中间值

2.Monitor训练状态监控数据采集位置如下

```
> verl > workers > megatron_workers.py
class ActorRolloutRefWorker(MegatronWorker, DistProfilerExtension):
    ):
        elif self._is_ref:

        # TODO: add more optimizer args into config
        if self._is_actor:
            optim_config_megatron = init_megatron_optim_config(
                optim_config,
                use_distributed_optimizer=wrap_config.use_distributed_optimizer,
                fp16=self.dtype == torch.float16,
            )
            ###monitor采集位置
            from msprobe.pytorch import TrainerMon
            monitor = TrainerMon(
                config_file_path = "/path/to/monitor.json",
                params_have_main_grad=True
            )
            monitor.set_monitor(
                actor_module,
                grad_acc_steps=num_micro_batch_size,
                optimizer=actor_optimizer
            )
            ###
            actor_optimizer = get_megatron_optimizer(model=actor_module, config=optim_config_megatron)
            actor_optimizer_scheduler = get_megatron_optimizer_param_scheduler(
                optimizer=actor_optimizer, config=optim_config
            )
        else:
            optim_config = None
            actor_optimizer = None
            actor_optimizer_scheduler = None

        log_gpu_memory_usage("After actor optimizer init", logger=logger)

        register_megatron_training_hooks(actor_module, actor_optimizer)
```

3.dump数据参照分级可视化工具或精度比对工具进行分析，若仍然没有定界到可疑算子，可以使用精度预检工具进行辅助排查。

3.4 不稳定复现问题定位

不稳定复现问题的关键点在于开启blocking、加同步之后，问题消失导致无法直接定位，目前经典案例为tensordict数据异步问题。

3.5 训推一致性问题

3.5.1问题背景及现象

在强化学习领域，目前主流RL算法是基于On-Policy前提展开的，On-Policy理论要求采样数据的行为策略与梯度计算的目标策略保持一致，才能确保梯度估计是无偏的，朝则梯度最陡峭的方向优化，使得训练过程更平稳。关于On-Policy和Off-Policy的区别，On-Policy就是与环境交互产生数据的策略和要更新的策略是同一个策略，Off-Policy就是两者策略存在不同。以PPO为例，Clipped目标函数为：

$$\mathcal{J}_{\text{PPO}}(\theta) = \mathbb{E}_{(s_t,a_t)\sim\pi_{\theta_{\text{old}}}}\left[\min\left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta),1-\epsilon,1+\epsilon)\hat{A}_t\right)\right] \qquad r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

$$\mathbb{E}_{a\sim\pi_{\theta_{\text{old}}}}\left[\nabla_\theta \min\left(\frac{\pi_\theta(a)}{\pi_{\theta_{\text{old}}}(a)}\hat{A},\ \text{clip}\left(\frac{\pi_\theta(a)}{\pi_{\theta_{\text{old}}}(a)},\ 1-\epsilon,\ 1+\epsilon\right)\hat{A}\right)\right]$$

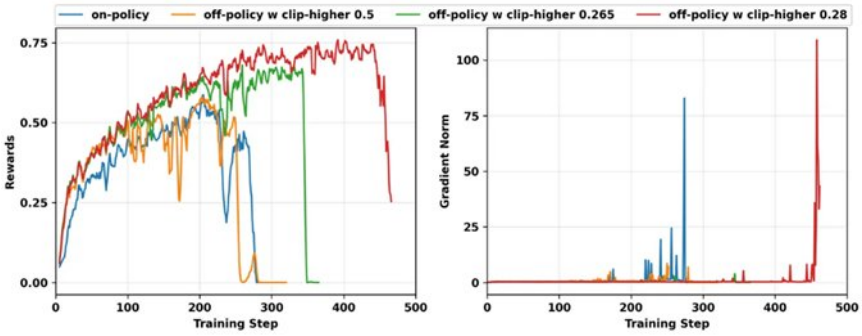
$$\mathbb{E}_{a\sim\pi_{\text{sampler}}(\theta_{\text{old}})}\left[\nabla_\theta \min\left(\frac{\pi_{\text{learner}}(a,\theta)}{\pi_{\text{sampler}}(a,\theta_{\text{old}})}\hat{A}, \text{clip}\left(\frac{\pi_{\text{learner}}(a,\theta)}{\pi_{\text{sampler}}(a,\theta_{\text{old}})}, 1-\epsilon, 1+\epsilon\right)\hat{A}\right)\right]$$

目前主流高效的RL训练框架中（VeRL、ms-Swift等），训推一般采用不同的引擎。通常会采用像vLLM、SGLang这样高度优化的推理引擎做数据采样，FSDP或Megatron等框架进行模型训练。

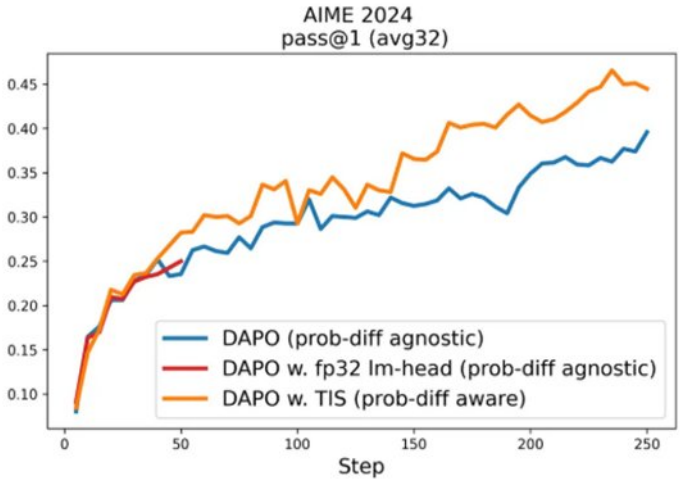
- 采样策略 *π_sampler*：推理引擎负责生成采样数据。
- 目标策略 *π_learner*：训练引擎负责计算梯度并更新模型参数。

On-Policy RL系统要求采样分布*π_sampler*与梯度计算的目标策略*π_learner*需要一致，由于训推存在差异，例如典型情况为**训推算子实现精度不一致、训推采用的量化精度FP8/BF16不一致，异步强化学习算法中训推模型不处于同一状态**等，使得算法潜藏Off-Policy的“陷阱”，不满足无偏估计的前提，导致出现以下两种典型场景。

场景1： 训练不稳定，训练过程直接出现Reward崩溃情况，图例为训练Reward突然崩溃情况，grad norm爆炸情况。



场景2： 收敛效果差，无法朝梯度最佳方向优化。图例为添加了TIS(Truncated Importance Sampling)取得了更优的效果



在两种场景中，训练崩溃场景更加重要，将直接导致训练无法继续进行，对于目前主流的方案和技术细节可参阅：强化学习的训推一致性问题

3.5.2一致性指标

在VeRL框架中，除了直接观测Reward、Grad Norm等异常指标，目前集成了简单的训推分布一致性统计值指标：

```
def calculate_debug_metrics(data: DataProto) -> dict:
    """
    calculate rollout vs actor logprobs diff, for debugging purpose

    Args:
        data: DataProto
```

```
        the data batch to calculate
        rollout_log_probs: log_probs record when rollout forward tokens
        old_log_probs(actor log probs): log_probs record when actor forward tokens
        loss_mask or attention_mask: to mask unrelated token
        responses: the response tokens, for calculating size
    Returns:
      dict: metrics
        "training/rollout_probs_diff_valid": 1->input is valid, 0->input is invalid
        "training/rollout_probs_diff_max": max value of logprob diff of rollout vs. actor
        "training/rollout_probs_diff_mean": mean value of logprob diff of rollout vs. actor
        "training/rollout_probs_diff_std": std value of logprob diff of rollout vs. actor
        "training/rollout_actor_probs_pearson_corr": logprob's pearson corrcoef of rollout vs. actor, refer
ence to https://arxiv.org/pdf/2506.13585
    """

    rollout_old_log_probs = data.batch["rollout_log_probs"]
    actor_old_log_probs = data.batch["old_log_probs"]
    if "response_mask" in data.batch:
      logger.debug("response mask found, use it to mask log probs")
      log_prob_mask = data.batch["response_mask"]
    elif "attention_mask" in data.batch:
      log_prob_mask = data.batch["attention_mask"]
    else:
      logger.warning(f"no mask info found, use all log probs, {(data.batch.keys())=}")
      log_prob_mask = torch.ones_like(rollout_old_log_probs)
    responses = data.batch["responses"]
    response_length = responses.size(1)

    response_mask = log_prob_mask[:, -response_length:]
    # calculate pearson corrcoef
    actor_probs = torch.exp(actor_old_log_probs)
    rollout_probs = torch.exp(rollout_old_log_probs)
    response_mask_bool = response_mask.bool()
    pearson_corrcoef = pearson_correlation_coefficient(actor_probs, rollout_probs, response_mask_bool)
    rollout_probs_diff = calculate_log_prob_diff(actor_probs, rollout_probs, response_mask_bool)
```

```
return {
    "training/rollout_probs_diff_valid": 1,
    "training/rollout_probs_diff_max": torch.max(rollout_probs_diff).detach().item(),
    "training/rollout_probs_diff_mean": torch.mean(rollout_probs_diff).detach().item(),
    "training/rollout_probs_diff_std": torch.std(rollout_probs_diff).detach().item(),
    "training/rollout_actor_probs_pearson_corr": pearson_corrcoef,
}
```

指标包括：

- rollout_probs_diff_max：训推的数值差异最大值
- rollout_probs_diff_mean：训推的数值差异平均值
- rollout_probs_diff_std：训推的数值的标准差
- rollout_actor_probs_pearson_corr：训推分布的皮尔森相关系数，越接近一表示两个分布更加接近。

目前只在在依赖于PPO ray trainer `verl\trainer\ppo\ray_trainer.py`中保有该输出，如DAPO ray trainer已经移除，若要开启该参数指标，需在启动脚本中添加：

```
actor_rollout_ref.rollout.calculate_log_probs=True
```

在0.7x最新代码中，集成了高阶的Off-Policy观测指标，开启上述配置后输出：

```
def compute_offpolicy_metrics(
    old_log_prob: torch.Tensor,
    rollout_log_prob: Optional[torch.Tensor],
    response_mask: torch.Tensor,
) -> dict[str, Any]:
    """Compute off-policy diagnostic metrics (helper function).

    This helper function operates on raw tensors and is used internally by:
    - compute_rollout_correction_and_rejection_mask() in this module (automatically included)
    - Tests (test_rollout_corr.py, test_rollout_corr_integration.py)

    These metrics help diagnose the off-policy gap between rollout and training policies,
    which can arise from:
    - Policy mismatch (e.g., vLLM BF16 vs FSDP FP32)
    - Model staleness (training on trajectories from older checkpoints)
    - General distribution shifts
```

```
    Key metrics:
    - kl: Direct KL divergence estimator KL(π_rollout || π_training)
    - k3_kl: K3 KL estimator for stability (more stable for small KL)
    - training_ppl: Perplexity of training policy
    - rollout_ppl: Perplexity of rollout policy
    - log_ppl_diff: Difference in log perplexities
    - ppl_ratio: Ratio of training PPL to rollout PPL
    - chi2_token: Token-level χ² divergence E[ρ²] - 1
    - chi2_seq: Sequence-level χ² divergence E[(Πρ_t)²] - 1

    Args:
        old_log_prob: Log probabilities from training policy, shape (batch_size, seq_length)
        rollout_log_prob: Log probabilities from rollout policy, shape (batch_size, seq_length)
        response_mask: Mask for valid tokens, shape (batch_size, seq_length)

    Returns:
        Dictionary of off-policy metrics (without prefix)
    """
    # Validate that we have at least one valid token
    assert response_mask.any(), "Expected at least one valid token in response_mask"

    metrics = {}

    # 1. Training policy perplexity (always available)
    # Formula: exp(-1/|T| * Σ log π_training(y_t|y_<t))
    # where |T| is the number of tokens generated by the model
    mean_log_prob_training = verl_F.masked_mean(old_log_prob, response_mask, axis=-
1)  # (batch_size,)
    training_ppl = torch.exp(-mean_log_prob_training).mean()  # Batch mean of per-sequence PPL
    metrics["training_ppl"] = training_ppl.detach().item()

    # Also log log-ppl for easier analysis (avoids exponential scale)
    metrics["training_log_ppl"] = (-mean_log_prob_training).mean().detach().item()
```

```
    # 2. Compute rollout off-policy metrics (only if rollout_log_probs available)
    if rollout_log_prob is not None:
        # 2a. kl: Direct estimator for KL(π_rollout ‖ π_training)
        # This is the standard KL divergence: E[log(π_rollout) - log(π_training)]
        # Positive value means rollout policy is more confident than training policy
        metrics["kl"] = verl_F.masked_mean(rollout_log_prob - old_log_prob, response_mask).detach().item(
)

        # 2b. k3_kl: K3 estimator for KL(π_rollout ‖ π_training)
        # More stable for small KL values using: E[exp(log_ratio) - log_ratio - 1]
        # Formula: KL ≈ E[r - log(r) - 1] where r = π_training/π_rollout
        log_ratio = old_log_prob - rollout_log_prob
        k3_kl_matrix = torch.exp(log_ratio) - log_ratio - 1
        metrics["k3_kl"] = verl_F.masked_mean(k3_kl_matrix, response_mask).detach().item()

        # 2c. Rollout policy perplexity
        mean_log_prob_rollout = verl_F.masked_mean(rollout_log_prob, response_mask, axis=-
1)  # (batch_size,)
        rollout_ppl = torch.exp(-mean_log_prob_rollout).mean()  # Batch mean of per-sequence PPL
        metrics["rollout_ppl"] = rollout_ppl.detach().item()
        metrics["rollout_log_ppl"] = (-mean_log_prob_rollout).mean().detach().item()

        # 2d. Log PPL difference (sequence-level perplexity difference)
        # log_ppl_diff = mean_log_prob_rollout - mean_log_prob_training
        # Since ppl = exp(-log_prob), we have:
        #   log(ppl_ratio) = log(training_ppl/rollout_ppl) = log_ppl_diff
        # Positive value means training assigns lower probability (higher PPL) than rollout
        log_ppl_diff = mean_log_prob_rollout - mean_log_prob_training
        metrics["log_ppl_diff"] = log_ppl_diff.mean().detach().item()
        metrics["log_ppl_abs_diff"] = log_ppl_diff.abs().mean().detach().item()
        metrics["log_ppl_diff_max"] = log_ppl_diff.max().detach().item()
        metrics["log_ppl_diff_min"] = log_ppl_diff.min().detach().item()

        # 2e. PPL ratio (how much higher is training PPL vs rollout PPL)
        # IMPORTANT: Compute per-sequence ratio first, then average
```

```
    # For numerical stability, compute in log space using log_ppl_diff
    # Note: log_ppl_diff = log(ppl_ratio), so ppl_ratio = exp(log_ppl_diff)
    # This is the inverse of geometric IS: ppl_ratio_i = 1 / geometric_is_i for each sequence
    ppl_ratio = torch.exp(log_ppl_diff).mean()  # mean(exp(log_ppl_diff)) = mean(ppl_ratio_i)
    metrics["ppl_ratio"] = ppl_ratio.detach().item()

    # 2f. Chi-squared divergence: χ²(π_training ‖ π_rollout) = E_μ[ρ²] - 1
    # where ρ = π_training / π_rollout and μ = π_rollout (rollout distribution)
    # This measures the variance of importance sampling weights
    # Token-level: E_token[ρ²] - 1 (averaged over all tokens)
    log_ratio_safe = torch.clamp(log_ratio, min=-SAFETY_BOUND, max=SAFETY_BOUND)
    rho_token = torch.exp(log_ratio_safe)  # ρ = π_training / π_rollout (token-level)
    rho_squared_token = rho_token.square()
    chi2_token = verl_F.masked_mean(rho_squared_token, response_mask) - 1.0
    metrics["chi2_token"] = chi2_token.detach().item()

    # Sequence-level: E_seq[(Π ρ_t)²] - 1 = E_seq[exp(2 * Σ log ρ_t)] - 1
    log_ratio_sum = verl_F.masked_sum(log_ratio, response_mask, axis=-1)  # Σ log ρ_t per sequence
    log_ratio_sum_safe = torch.clamp(log_ratio_sum, min=-SAFETY_BOUND, max=SAFETY_BOUND)
    rho_squared_seq = torch.exp(2.0 * log_ratio_sum_safe)  # (Π ρ_t)²
    chi2_seq = rho_squared_seq.mean() - 1.0
    metrics["chi2_seq"] = chi2_seq.detach().item()

  return metrics
```

关键指标包括：

- kl: 训推分布的KL散度
- k3_kl: 训推分布的K3_KL散度
- training_ppl: 训练分布的PPL困惑度
- rollout_ppl: 推理分布的PPL困惑度
- log_ppl_diff: 训推PPL的对数差异
- ppl_ratio: 训推PPL的比值
- chi2_token: Token级的概率卡方分布
- chi2_seq: Sequence级的概率卡方分布

3.5.3使能Rollout correction

VeRL框架在经过多轮重构后，目前实现了Rollout correction的一整套算法，包括:

- **重要性采样加权（Importance Sampling weights, IS）**
- **拒绝抽样（Rejection sampling, RS）**
- **否决机制（veto mechanism）**

Yaml Config使能方式:

```
algorithm:
  rollout_correction:
    rollout_is: token                    # IS权重计算方式：可选"token"（按token计算）、"sequence"（按序列计算）或null（不使用）
    rollout_is_threshold: 2.0            # IS权重的上阈值，用于限制权重的最大值
    rollout_is_batch_normalize: false    # 是否对IS权重进行批归一化，使其均值为1.0
    rollout_rs: null                     # 拒绝采样策略：可选"token"、"sequence"、"geometric"或null
    rollout_rs_threshold: null           # 拒绝采样的上阈值，使用RS时必须设置
    rollout_rs_threshold_lower: null     # 拒绝采样的下阈值，若未指定则自动设为上阈值的倒数
    rollout_token_veto_threshold: null   # 每个token的否决阈值，若为null则不启用
    bypass_mode: false                   # 是否跳过旧策略概率的计算
    use_policy_gradient: false           # 是否使用策略梯度损失（而非PPO损失）

# 必填项：启用对数概率计算
actor_rollout_ref:
  rollout:
    calculate_log_probs: true            # 是否计算对数概率，策略优化算法通常需要此步骤
```

目前重要性加权、拒绝采样等策略，在昇腾上还未有具体的实践案例和明确效果影响分析结论。在项目在如涉及到训推不一致问题，可以尝试开启IS或者RS等措施，具体参数配置业界案例在VeRL上有已经探索的配置，可参考：VeRL rollout correction说明

4. RL精度问题典型案例

4.1 checklist不一致案例

4.1.1 模型配置未对齐

【字节】tokenizer_config未对齐导致response异常

背景：

Qwen3-8B模型reward与response length与GPU差异大，精度无法对齐

定位过程：

参数、代码版本与GPU对齐后，对GPU与NPU的权重文件进行md5校验，发现tokenizer_config.json 文件md5值存在差异，进一步对比发现npu的tokenizer config中chat_template去除了使能思考（think）的字段，导致模型推理输出长度缺少思考过程，造成精度差异

根因&解决方案：

将tokenizer_config.json文件还原为原始状态。

结果：

修复问题后，reward与response length基本对齐

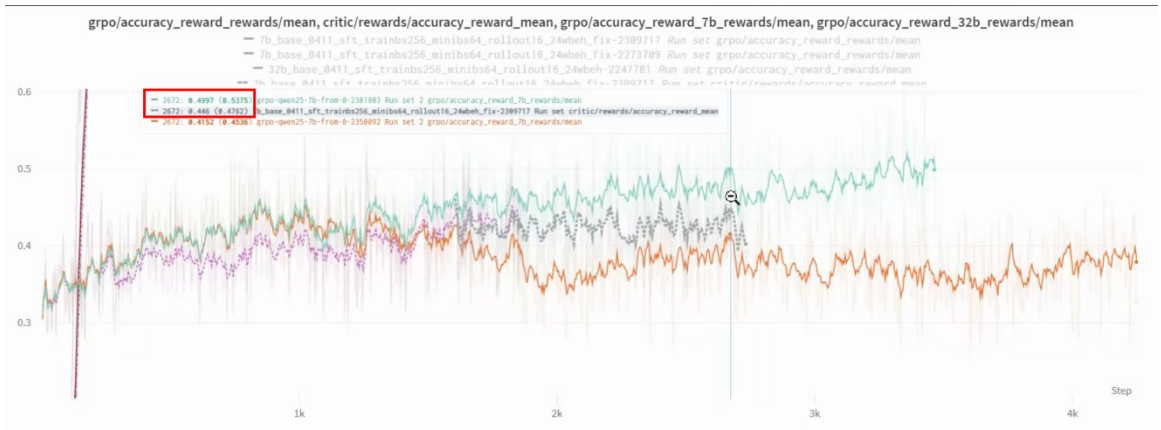
案例链接：Wiki: 【ZJ】Qwen3-8B精度问题定位

4.1.2 训练超参未对齐

【唯品会】RMSNorm超参未对齐导致reward下降

背景：

Qwen2.5-7b长跑1000步训练reward和GPU对齐（略高于GPU），但之后缓慢下降并显著低于GPU（橙色是问题NPU、粉色是GPU基线、绿色是修复后的NPU）



定位过程：

1）使用相同的权重在GPU和NPU上用客户的vllm评测脚本进行评测，推理结果正常，评测结果对齐，说明推理部分无精度问题

2）在rollout打桩且input ids已经完全对齐的前提下，插入hook打印对比训练阶段的actor前反向精度，发现经过layernorm层后，NPUdump的结果和GPU出现较大差异

3）排查layernorm配置，定位到在跨框架配置对齐时，mindspeed-rl中的rms_norm_eps和客户的未对齐。mindspeed-rl中如果不进行配置，使用的是默认是1e-6，但客户使用的是1e-7。（已提问题单，版本现已修改）

根因&解决方案：

手动在yaml中配置rms_norm_eps为1e-7即可对齐

结果：

长跑4k步训练loss和GPU对齐，缓慢上升，不发生下降

案例链接：Wiki: 【唯品会】MindSpeed-RL精度问题

4.2 确定性固定不住案例

【京东】verl随机性固定不住

背景：

Qwen2.5-72B模型，使用GRPO算法+megatron训练后端，A3和A2之间reward表现存在较大差异，A3训练过程中频繁出现reward尖刺，需要定位精度问题

定位过程：

1、代码环境和版本对齐。和开发对齐当前版本存在以下已知问题：

- tensordict问题，需要升级到tensordict=0.10.0
- cann需要升级到630补丁版本

更新tensordict后，reward尖刺消失。

2、开启确定性进行精度对齐

1）检查确定性使能：使用msprob开启确定性计算，在函数入口添加了seed_all代码。

2）减层复现，采集dump：减到4层后现象一致，输入仍然无法固定。使用dump工具采集两次运行的整网各算子输入输出的md5结果进行对比（dump的位置：update，训练的输入就是推理的输出，只要确认一个位置对齐就可以了），发现前向能够完全固定，最后一层FA的反向首次出现不对齐，因此怀疑FA算子未生效确定性。

3）单算子验证：采集整网最后一层FA算子的输入输出tensor数据进行单算子确定性测试，单测用例中FA跑多次结果完全一致，单测FA能够完全固定，但是整网的grad计算结果和单测的grad计算结果不同。因此怀疑整网未生效，进一步进行整网的FA TilingKey确认，**发现整网的FA未使能确定性**。（验证FA是否生效可以参考案例链接）

4）框架侧使能确定性验证：通过plog查询结果发现整网的确定性计算未生效，因此基本确认是框架侧确定性计算使能方式有问题

根因&解决方案：

VeRL等使用Ray进行分布式训练的框架，由于ray集群在启动时会将环境变量作为快照固定下来，快照一经生成即只读，后续再修改不会同步到已运行的raylet，必须通过runtime_env显式覆盖，这个值会在创建actor进程的时候传入到子进程中。因此如果只在主函数入口添加确定性，无法传递到worker中。由于训练的前反向过程都在worker中进行，因此需要在worker中添加seed_all

结果：

正确使能确定性后，训练情况如下，20步 grad结果完全一致

案例链接：Wiki: 【JD】使用verl随机性固定不住问题

4.3 稳定复现案例

4.3.1 训练精度问题

【蚂蚁】Ring-mini TE+ETP1梯度爆炸

背景：

在asys框架下训练ring-mini，不开启TE时精度对齐；修改训练配置，开启TE且etp=1的情况下，大概率第1步出现梯度爆炸

定位过程：

1）修改训练配置后精度未对齐，说明是训练精度问题。打桩rollout输出，可以跳过推理阶段，快速复现精度问题。

2）采用mstt monitor工具定位到layers.6.mlp.share_experts.linear_fc1.weight梯度异常。

3）task_queue_enable=0不复现，怀疑发生多流内存踩踏，_sanitizer流同步检测工具检测，发现npu_matmul_add_fp32和run_backward可能发生多流内存踩踏

4）工具打印的npu_matmul_add_fp32调用栈发现，npu_matmul_add_fp32用于gradient_accumulation_fusion特性打开的代码分支，关闭该特性后多次实验不复现

5）MindSpeed同事怀疑是moe_shared_expert_overlap问题，采集profile分析验证，发现反向存在2条流，且共享专家反向的算子不在计算主流stream 2上，建议关闭参数moe_shared_expert_overlap，关闭后解决

根因&解决方案：

该问题之前MindSpeed已经修复过，问题单为：https://dts-szv.clouddragon.huawei.com/DTSPortal/ticket/DTS2025060624230，但该问题又复现了，怀疑之前的修复方案存在遗漏场景，在特定场景/时序下会触发；现场通过关闭moe_shared_expert_overlap规避

结果：

关闭moe_shared_expert_overlap后梯度爆炸问题不复现，版本仍在定位中

案例链接：Wiki: 【MY】Ring-mini TE+ETP1梯度爆炸问题定位

4.3.2 推理精度问题

【字节】开关虚拟内存导致kv cache驱逐

背景：

Qwen2.5 32B开关虚拟内存长跑reward和response length出现分叉问题

定位过程：

1）缩小规模、减小response length为2k后，打开确定性计算，开关虚拟内存后模型response完全一致，验证了虚拟内存的开关并不影响确定性；

2）回升规模，开确定性计算，发现开关虚拟内存、开虚拟内存连续跑两次的response结果不完全一致；

3）采集推理过程的L0层级数据，发现从第484个token开始，shape会对不上，之前的所有数据完全一致：

4）找到第一个shape出现差异的token，通过比对结果中shape对不上的地方倒查调用栈，追溯发现是input_batch.num_reqs不同从而引发差异；进一步定位，发现是由于显存不足导致驱逐

根因&解决方案：

vllm推理过程中，当显存申请不足时 Scheduler 会**“驱逐”**掉一部分请求，以保证推理的正常进行；由于开关虚拟内存导致剩余显存不一致，引发vllm显存管理时驱逐的行为不一致，最终导致精度差异；通过打桩剩余显存的方式，让开关虚拟内存的剩余显存对齐后response完全一致。

结果：

在vllm中开启采样的情况前提下，response会受随机数影响，要固定确定性除了要固定采样精度外，还需要对齐显存以对齐调度过程。

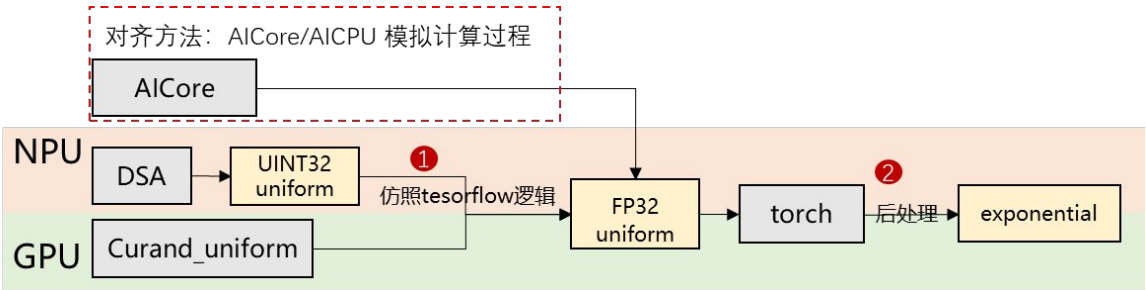
案例链接：Wiki: 【ZJ】verl Qwen2.5 32B 开关虚拟内存精度对不齐问题

【字节】dsa uniform带来的指数分布差异

背景：

vllm推理首token，vllm随机采样实现中，调用了torch的exponential_算子，而由于GPU和NPU在随机数算子底层实现上存在诸多差异，因此无法实现与GPU的逐token对齐。

定位过程



如上图，GPU、NPU均是先生成FP32的均匀分布随机数（uniform），然后再由torch框架进行后处理，生成指数分布的随机数，差异主要是以下两点：

① **dtype转换差异：**NPU是通过DSA硬件的随机数生成器（电路编码在硬件上）先生成UINT32格式的均匀分布随机数，然后再仿照tensorflow逻辑转换成FP32的dtype，这里与GPU的curand_uniform存在差异

② **后处理差异：**torch框架层中，offset计算的increment、开闭区间转换等操作也与GPU存在差异，共同造成了指数分布的差异

解决方案

为对齐上述差异，目前的解决方案为直接使用AICore/AICPU去模拟curand_uniform的计算过程，并直接在AICore中进行uniform的后处理操作，但会带来严重的性能劣化，尽管使用AICore进行计算，依然会有10倍的性能劣化，目前的解决方案是在推理框架侧（vllm-ascend）额外创建一条计算流，提前计算exponential来掩盖掉该段计算。使用AICPU计算的方案无法实现完全掩盖，而AICore方案时间与dsa相近，但由于使用AICore计算，还是会抢占计算资源，在计算压力较大的场景下多流无法实现太好的掩盖，目前可以在修复exponential算子的同时端到端性能不劣化

实验效果

在math500评测集的500道题中进行单推测试，未修改exponential算子实现的NPU与GPU输出的首个token对齐率仅73/500，在对齐了指数分布计算逻辑后，对齐率达到498/500，首token之后平均能对齐前150+个token，对齐率大幅提升（数据来源 - 精度问题分析）

案例链接：

4.3.3 训推转换问题

【电信】Deespeek训推sharding精度问题

背景：

Deepseek-671b推理设置vllm的load_format为dummy（即随机初始化）时，NPU侧的推理始终为乱码，但GPU能够正常推理

定位过程：

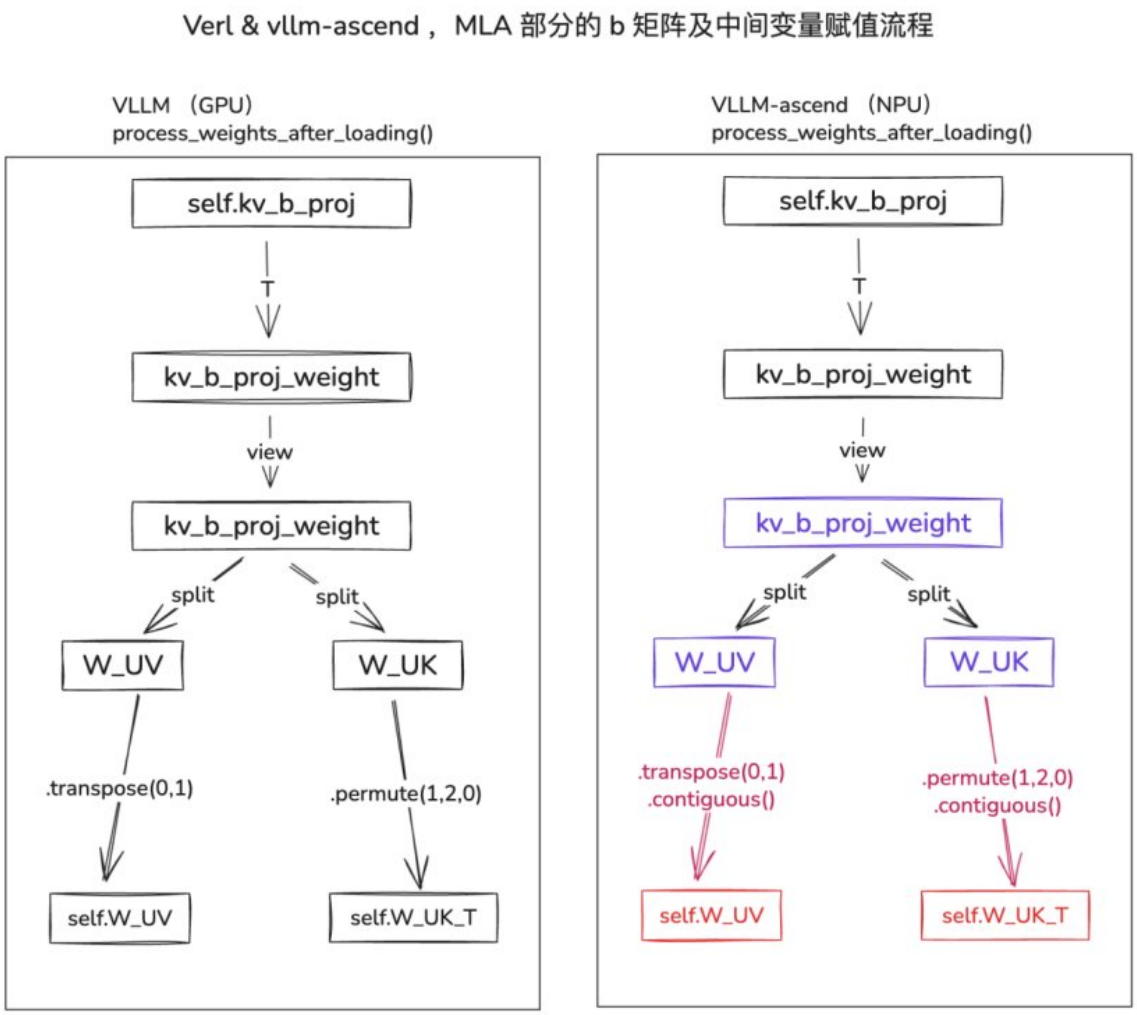
1）由于资源限制，先在Qwen3-30B（MoE）、Qwen3-32B（Dense）上实验没有复现（没开mc2没开nz），load_format为dummy时推理正常，说明当前的verl代码的load_format=dummy能力是具备的

2）在类DeepSeek的Moonlight-16B模型上**可以复现该问题**，但对比load_format为safetensors和dummy两者sharding后的权重内容，发现权重是一致的，因此需要打开细看。

3）由于load_format=dummy时，GPU没问题但NPU有问题，因此可以进一步通过前向对比推理的数据，通过一层一层打印每一个计算步骤的输入输出。这时候定位到前向不一致的位置在kv_b矩阵相关计算的地方存在差异。更具体的存在差异的是

W_UV、W_UK_T矩阵。通过走读代码，找到这个矩阵的来源，发现权重sharding后，这两个矩阵没有被改变。

根因&解决方案：



dummy 权重初始时候是随机初始化权重，初始根据模型结构获取到的 self.kv_b_proj 的权重在赋值给 kv_b_proj_weight 是正确的，但后续经过了 view 操作再 split 生成前向计算所需要的 W_UV、W_UK_T 时拿到的权重和原来的kv_b_proj_weight矩阵的视图被打断（地址发生变化），强化学习的训推转换做sharding时，只对kv_b_proj_weight做了in-place替换操作，但是W_UV、W_UK_T没有被更新。

这部分导致在 vLLM-ascend 在 dummy 下精度出现错误，但本质上影响了整个训推流程。

解决方案:

1）在加载完权重后暴力添加 process_weights_after_loading，这样不论怎么样权重都是会更新的，适用于 veRL 中的 update_weights 后的行为（已合入verl仓）

2）修复此处的 view 操作，使其行为与 GPU 保持一致；并删除.contiguous()操作（会影响性能）

结果:

修复后load_format为dummy时NPU侧能正常推理吐字

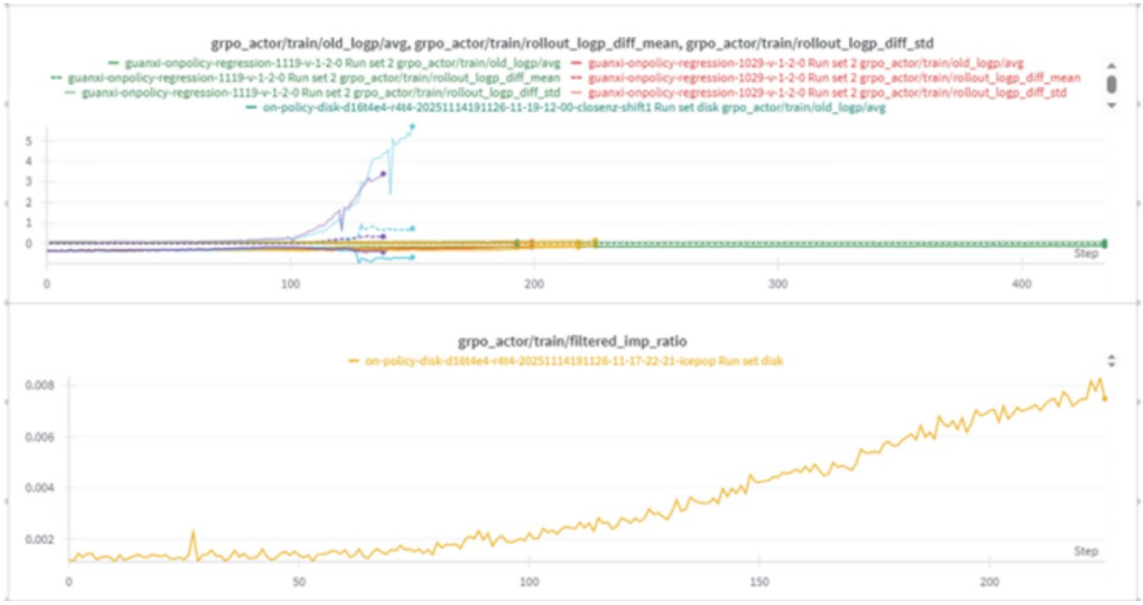
案例链接：Wiki: Verl DeepSeek 推理load_format=dummy

4.3.4 训推一致性问题

【蚂蚁】ring mini开NZ训推算子不一致问题

背景：

ring mini长跑100步后，reward下降，logp diff和logp std出现明显的抬头现象出现明显训推不一致特征，gradnorm出现爆炸情况



定位过程：

1）现场发现NPU打开NZ数据排布的情况下在接近100步时出现训崩，关闭后有所缓解，现最多训至300+step暂未发现reward开口现象。

2）GPU侧也有训推不一致问题，但是大约在400step时才会复现该问题，并可以通过icepop手段拖延至1200步后才出现差异。技术文档链接：Small Leak Can Sink a Great Ship—Boost RL Training on MoE with !

根因&解决方案：

该问题的根因为训推不一致，NZ（为满足AICore中Cube计算单元高性能计算的需要，引入该特殊格式）影响了数据排布，把训推不一致的问题放大了；目前蚂蚁侧GPU解决方案为直接用训练算子替换推理算子进行计算，

NPU侧通过VLLM_ASCEND_ENABLE_NZ控制NZ开关，PR：[Feat] Unquantized Linear to nz and control all nz-cast (#3356) · vllm-project/vllm-ascend@07e3962 · GitHub

结果：

关闭NZ后训崩和训推不一致的问题能够缓解

案例链接：Wiki: Asystem ring 1T/mini NZ数据排布格式导致训推算子不一致问题总结

4.4 不稳定复现案例

【京东】tensordict数据异步问题

背景：

长跑时发现grad_norm等指标出现异常现象，NPU端grad_norm整体大于GPU端，并且长跑过程中出现激增的现象，存在大量尖刺

定位过程：

1）开启确定性，打桩采集完整的前反向数据，将打桩数据插入到推理阶段模型前向函数之后，该现象消失。判断问题出在前反向阶段

2）通过排查前反向阶段的输入输出，确认精度问题由第二个step起ref计算结果错误引起，不添加同步会导致ref_logp值和old_logp完全一致；添加同步后ref值恢复正常

3）使用tensor.data_ptr排查内存地址，发现ref_logp和old_logp在npu上是两个内存地址，kl_penalty在host侧计算，但是这两个数据在cpu上变成同一个地址了。因此怀疑问题出在tensordict.to这个接口

根因&解决方案：

tensordict三方库没有做原生适配，但是在verl中使用到了。其中tensordict.to方法默认是异步的，会根据使用情况加上同步，但是由于没有对npu做原生适配，npu上torch.cuda.is_initialized始终为False，导致未正确进行同步操作。由于异步操作加上内存地址复用，在verl中ref worker的返回值和actor 返回值的指针地址一致，从而导致ref worker错误地、提前返回了actor worker的计算结果，造成内存踩踏。

解决方案：升级tensordict==0.10.0

结果：

添加同步操作后，训练效果正常。

案例链接：Wiki: 【JD】tensordict原生适配npu导致精度问题

5. 工具使用

5.1 msprobe dump工具

介绍：msprobe 工具主要通过在训练脚本内添加 dump 接口、启动训练的方式采集精度数据。

链接：
https://gitcode.com/Ascend/mstt/blob/master/debug/accuracy_tools/msprobe/docs/zh/dump/pytorch_data_dump_instruct.md

5.2 monitor训练状态轻量化监控工具

介绍：Monitor训练状态轻量化监控工具，能够在较低性能损耗下收集和记录模型训练过程中的激活值、权重梯度、优化器状态和通信算子的中间值，实时呈现训练状态。

链接：
https://gitcode.com/Ascend/mstt/blob/master/debug/accuracy_tools/msprobe/docs/zh/monitor_instruct.md

5.3 npu_sanitizer工具

介绍：npu_sanitizer工具，对标CUDA Sanitizer工具，是一个用于检测在不同流上运行的内核之间同步错误的工具。它通过存储对张量的访问信息来判断这些访问是否已同步。当在 Python 程序中启用该工具并检测到可能的数据竞争时，将打印详细警告并终止程序。

链接：https://docs.pytorch.org/docs/stable/cuda._sanitizer.html

注：这是一个原型功能，意味着它处于早期阶段，供反馈和测试，并且其组件可能会发生变化。

使用方法：

在入口python中加入：

```
from torch_npu.npu import _sanitizer
_sanitizer.enable_npu_sanitizer()
```